Molecular Modeling Toolkit ドキュメント

リリース *0.1.0*

Molecular Modeling Development Team

2026 年 02 月 23 日

はじめに

第 1 章　概要　1

1.1　このツールキットでできること 1
1.2　処理パイプライン 1
1.3　Voronoi 最適化 2
1.4　置換機能 2
1.5　入出力 3
1.6　GUI 3

第 2 章　インストール　5

2.1　動作要件 5
2.2　インストール手順 5

第 3 章　やりたいこと別ガイド　7

3.1　結晶構造を見たい・開きたい 8
3.2　スーパーセルを作りたい 8
3.3　分子を追加・配置したい 8
3.4　原子・元素を置換したい 9
3.5　分子配向を最適化したい 9
3.6　構造をファイルに出力したい 10
3.7　表示を変えたい 10
3.8　原子を選択したい 11
3.9　操作を戻したい 11
3.10　TOML 設定ファイルの書き方を知りたい 11

第 4 章　クイックスタート　13

4.1　パイプラインの実行 13
4.2　入力ファイル 13
4.3　出力ファイル 14
4.4　GUI の起動 14
4.5　次のステップ 14

第 5 章　チュートリアル　17
5.1　ステップ 1: 分子定義の理解　17
5.2　ステップ 2: 入力ファイルの設定　18
5.3　ステップ 3: パイプラインの実行　19
5.4　ステップ 4: 出力ファイルの確認　19
5.5　ステップ 5: Voronoi 可視化の見方　20
5.6　ステップ 6: GUI でのインタラクティブ操作　20
5.7　ステップ 7: 置換機能の詳細　21

第 6 章　GUI 操作ガイド　23
6.1　起動方法　23
6.2　ウィンドウ構成　23
6.3　メニュー一覧　25
6.4　ビュー操作ツールバー　27
6.5　ステータスバー　29
6.6　キーボードショートカット　29
6.7　マウス操作　30
6.8　ダイアログ詳細　30

第 7 章　コマンドラインツール　37
7.1　run.py — パイプライン実行　37
7.2　molecular_modeling_gui.py — GUI 起動　38
7.3　vis_cif.py — CIF ビューア　38
7.4　edit_and_run.py — GUI 編集＋最適化　39
7.5　テストの実行　39

第 8 章　入出力ファイル形式　41
8.1　ファイル構成の概要　41
8.2　入力設定: input/input.toml　42
8.3　分子定義: data/<name>.toml　45
8.4　完全な設定例　48

付録 A 章 Voronoi 最適化の詳細　53
A.1　問題設定　53
A.2　Voronoi セルとは　53
A.3　IoU（Intersection over Union）　53
A.4　サイトごとの最適化手順　54
A.5　辞書の Voronoi セルの構築方法　56
A.6　体積タイリング検証　56
A.7　最適化結果の解釈　56
A.8　可視化（Voronoi オーバーレイ PNG）　57
A.9　計算に使用するライブラリ　57

付録 B 章 アーキテクチャ 59
B.1 パイプラインフロー 59
B.2 各モジュールの役割 60
B.3 system.info メタデータ 62
B.4 GUI アーキテクチャ 62

付録 C 章 API リファレンス 65
C.1 モジュール構成 65
C.2 メインパイプライン 65
C.3 システム構築 66
C.4 Voronoi ジオメトリ 67
C.5 最適化 74
C.6 可視化 76
C.7 出力 77
C.8 GUI 77

第

ONE

概要

Molecular Modeling Toolkit は、ASE (Atomic Simulation Environment) をベースとした分子モデリングツールキットです。TOML 形式の分子定義ファイルからスーパーセルを構築し、Voronoi セル IoU 最大化による分子配向の最適化を行い、Quantum ESPRESSO / VASP / CIF 形式で出力します。

1.1 このツールキットでできること

- **結晶構造の構築**: CIF ファイルの読み込み、スーパーセル作成、分子の追加・配置
- **分子配向の自動最適化**: 参照結晶との Voronoi セルの重なりを最大化して最適な向きを決定
- **原子・分子の置換**: 元素の一括変換、分子の入れ替え、個別原子の変更（4 種類の置換方法）
- **構造ファイルの出力**: Quantum ESPRESSO / VASP / CIF 形式への一括エクスポート
- **GUI での対話操作**: 3D 可視化、マウス操作、各種ダイアログによる直感的な編集

1.2 処理パイプライン

本ツールキットの中心となる処理は、6 段階のパイプラインで構成されています:

1. **構築** -- TOML 設定ファイルと分子定義からスーパーセルを組み立て、原子置換を適用
2. **グループ化** -- 原子を分子サイトごとにインデックス化し、以降の分子単位の操作を可能に
3. **最適化** -- Euler 角のグリッドサーチ + Nelder-Mead 法で Voronoi セル IoU を最大化
4. **検証** -- Voronoi 体積の合計がスーパーセル体積と一致するかチェック
5. **可視化** -- 3D Voronoi セルオーバーレイの描画 (最適化前後の比較 PNG を生成)
6. **出力** -- Quantum ESPRESSO, VASP, CIF 形式への一括エクスポート

```
input.toml + data/*.toml (分子定義)
     |
```

(次のページに続く)

(前のページからの続き)

```
     v
[1. 構築] スーパーセル生成 + 原子置換
     |
     v
[2. グループ化] 分子サイトごとの原子インデックス作成
     |
     v
[3. 最適化] Voronoi IoU 最大化（グリッドサーチ + Nelder–Mead）
     |
     v
[4. 検証] Voronoi 体積のタイリング整合性チェック
     |
     v
[5. 可視化] Voronoi オーバーレイ PNG 生成
     |
     v
[6. 出力] qe.in / POSCAR / INCAR / KPOINTS / structure.cif
```

1.3 Voronoi 最適化

本ツールキット独自の機能である Voronoi 最適化は、以下の手順で行われます:

- **Voronoi セルの構築**: 分子の重心座標から周期境界条件を考慮してセルを計算
- **体積スケーリング**: 系と辞書の Voronoi 体積の違いを均一スケールで補正
- **回転の最適化**: 13x13x13 の Euler 角グリッドサーチで粗い探索を行い、Nelder-Mead 法で精密化
- **評価指標**: IoU (Intersection over Union) = 交差体積 / 和集合体積 (0--1、高いほど良い)

詳細は *Voronoi* 最適化の詳細 を参照してください。

1.4 置換機能

4 種類の置換機能により、結晶構造を柔軟にカスタマイズできます。置換は以下の順序で適用されます:

1. **分子置換**: 特定のセル位置の分子を別の分子に丸ごと置換
2. **グローバル元素置換**: 系全体で特定の元素を別の元素に一括変換 (例: 全ての H を He に)
3. **分子内原子置換**: 特定の分子の原子リストを新しい元素リストで再定義 (グローバル置換の結果を上書き)
4. **個別原子置換**: 特定のセル・分子・原子インデックスで指定した 1 原子のみ置換

1.5 入出力

入力ファイル:

- `input/input.toml` -- 格子ベクトル、スーパーセル繰り返し数、分子リスト、重心座標、置換ルール
- `data/<name>.toml` -- 分子ごとの原子座標、格子ベクトル、重心、元素記号 (化学式の簡略記法 `"C6H6"` にも対応)

出力ファイル:

- `source/structure_out/qe.in` -- Quantum ESPRESSO 入力ファイル
- `source/structure_out/POSCAR`, `INCAR`, `KPOINTS` -- VASP 入力ファイル一式
- `source/structure_out/structure.cif` -- CIF 形式の結晶構造ファイル
- `source/voronoi_pngs/*.png` -- 分子サイトごとの Voronoi セルオーバーレイ可視化画像

1.6 GUI

ASE-GUI を拡張した Tkinter ベースの GUI を提供しており、コマンドライン操作とインタラクティブな操作の両方に対応しています。主な機能:

- CIF ファイルの読み込みと 3D 可視化
- 分子の追加・削除・置換をダイアログから対話的に実行
- セルパラメータの編集とスーパーセルの構築
- 最適化・可視化・エクスポートの一括実行
- 原子の選択、フォーカス表示、測定 (距離・角度・二面角)
- 元素・分子グループ単位での表示・色・サイズの制御

第

TWO

インストール

2.1 動作要件

- Python 3.11 以上（3.13 で動作確認済み）
- pip または uv パッケージマネージャ

2.1.1 依存ライブラリ

主要な依存ライブラリは以下の通りです（`pyproject.toml` で管理）：

- **ASE** (3.25.0) — Atomic Simulation Environment
- **NumPy** (2.3.1) — 数値計算
- **SciPy** (1.16.0) — Voronoi 計算、最適化
- **matplotlib** (3.10.3) — 可視化
- **trimesh** (4.7.1) — メッシュ操作
- **manifold3d** (3.2.1) — マニフォールド再構成
- **Pillow** (11.3.0) — 画像処理

2.2 インストール手順

2.2.1 1. リポジトリのクローン

```
git clone <repository-url>
cd molecular_modeling
```

2.2.2 2. 仮想環境の作成

```
python -m venv .venv
source .venv/bin/activate    # Linux / macOS
.venv\Scripts\activate       # Windows
```

2.2.3 3. パッケージのインストール

pip を使用する場合:

```
pip install -e .
```

uv を使用する場合:

```
uv sync
```

2.2.4 4. 動作確認

テストを実行して正しくインストールされたことを確認します:

```
pytest
```

全テストがパスすれば、インストールは完了です。

2.2.5 Conda 環境の場合

`environment.yml` を使用して Conda 環境を構築できます:

```
conda env create -f environment.yml
conda activate molecular_modeling
```

詳細は `CONDA_SETUP.md` を参照してください。

第

THREE

やりたいこと別ガイド

「こういうことがしたい」→「こうすればできる」の逆引きリファレンスです。詳細は各リンク先を参照してください。

カテゴリ

- 結晶構造を見たい・開きたい
- スーパーセルを作りたい
- 分子を追加・配置したい
- 原子・元素を置換したい
- 分子配向を最適化したい
- 構造をファイルに出力したい
- 表示を変えたい
- 原子を選択したい
- 操作を戻したい
- *TOML* 設定ファイルの書き方を知りたい

3.1 結晶構造を見たい・開きたい

やりたいこと	操作
CIF ファイルを開いて構造を見たい	GUI: **MoleculeTools → Load CIF...** またはコマンドラインで `python vis_cif.py file.cif` (詳細は コマンドラインツール を参照)
QE / VASP / XYZ など他の形式を開きたい	GUI: **MoleculeTools → Load Structure...** (Ctrl+L)。ASE が対応する形式ならすべて読めます
TOML プロジェクトを読み込みたい	GUI: **MoleculeTools → Load Project...** (Ctrl+T)。`input.toml` を選択すると分子グループ情報も復元されます
最近開いたファイルをすぐ開きたい	GUI: **Recent Files** メニューに最近の 10 件が表示されます

3.2 スーパーセルを作りたい

やりたいこと	操作
単位胞を繰り返してスーパーセルにしたい	GUI: **MoleculeTools → Build Supercell...** で [a, b, c] 方向の繰り返し数を指定。詳細は チュートリアル のステップ 2 を参照
セルパラメータ (a, b, c, α, β, γ) を確認・変更したい	GUI: **MoleculeTools → Cell Parameters...**
TOML で一括設定したい	`input.toml` の `supercell_repeat = [2, 2, 1]` で指定。詳細は 入出力ファイル形式 を参照

3.3 分子を追加・配置したい

やりたいこと	操作
スーパーセルに分子を追加したい	GUI: **MoleculeTools → Add Molecule...** (Ctrl+J)。`data/` フォルダの TOML 分子定義から選択し、位置と配向を指定。詳細は *GUI 操作ガイド* の Add Molecule ダイアログを参照
分子の位置を動かしたい	分子中の原子を選択 → **Selection → Move Molecule/Atoms...**。X/Y/Z 方向の変位を ± ボタンで調整
分子の向きを回転したい	分子中の原子を選択 → **Selection → Rotate Molecule/Atoms...**。トラックボールまたは Euler 角入力で回転
分子をマウスでドラッグ移動したい	**Selection → Toggle Drag Mode** で有効化。クリック＆ドラッグで直接移動
分子を削除したい	分子中の原子を選択 → **Selection → Delete Molecule...**
選択した原子を新しい分子として保存したい	原子を選択 → **MoleculeTools → Save Selection as Molecule...**。`data/` に TOML ファイルが作成されます

3.4 原子・元素を置換したい

やりたいこと	操作
系全体で特定の元素を一括置換したい（例: 全 H → D）	GUI: **MoleculeTools → Global Element Substitution...**。または `input.toml` の `[global_atom_substitutions]` で設定
特定の分子だけ別の分子に丸ごと入れ替えたい	GUI: **MoleculeTools → Molecule Substitution...**。置換先の分子を選択すると、PCA で配向が維持されます
1 つの原子だけ別の元素に変えたい	GUI: **MoleculeTools → Individual Atom Substitution...**
複数の元素置換をまとめて実行したい（例: C→Si, H→F を同時に）	GUI: **MoleculeTools → Batch Element Substitution...**。複数のルールを一度に指定できます
特定の分子の全原子を新しい元素リストで再定義したい	GUI: **MoleculeTools → Molecule Atom Substitution...**

3.5 分子配向を最適化したい

やりたいこと	操作
Voronoi 最適化で分子配向を自動決定したい	GUI: **Pipeline → Run Optimize (Voronoi)** (Ctrl+P)。参照結晶との Voronoi セルの重なりを最大化します。アルゴリズムの詳細は *Voronoi 最適化の詳細* を参照
最適化結果（IoU・回転角・スケール）を確認したい	GUI: **Pipeline → Show Optimization Results...** (Ctrl+H)。CSV エクスポートも可能です
Voronoi セルの重なりを画像で確認したい	GUI: **Pipeline → Run Render (Voronoi PNGs)...**。青＝系、赤＝辞書、オレンジ＝交差部分
最適化から出力まで一括実行したい	GUI: **Pipeline → Run All (Optimize → Render → Export)...**
コマンドラインで一括実行したい	`python run.py`。詳細は コマンドラインツール を参照

3.6 構造をファイルに出力したい

やりたいこと	操作
Quantum ESPRESSO の入力ファイルを作りたい	GUI: **Export → Quantum ESPRESSO...** または **Export → Export All (QE, VASP, CIF)...** (Ctrl+D)
VASP (POSCAR/INCAR/KPOINTS) を作りたい	GUI: **Export → VASP...** または **Export → Export All...**
CIF ファイルとして保存したい	GUI: **Export → CIF...** または **Export → Export All...**
3 形式すべてをまとめて出力したい	GUI: **Export → Export All (QE, VASP, CIF)...** (Ctrl+D)
プロジェクト (TOML) として保存したい	GUI: **MoleculeTools → Save Project...** (Ctrl+K)。`input.toml` と `data/*.toml` が保存されます
3D ビューのスクリーンショットを保存したい	GUI: **View Settings → Save Screenshot...** (Ctrl+Shift+S)

3.7 表示を変えたい

やりたいこと	操作
結合（ボンド）を表示したい	**View Settings → Show Bonds** (Ctrl+B) で切り替え。サイドパネルのチェックボックスでも操作可
水素原子を非表示にしたい	**View Settings → Toggle Hydrogen**
特定の元素だけ表示/非表示したい	**View Settings → Element Visibility...**
分子ごとに色分けしたい	**View Settings → Color by Molecule Group**
原子の色・サイズを変えたい	**View Settings → Atom Appearance...**
背景色を変えたい	ツールバーの **BG Color** ボタン（白/黒/ダーク/ライト/カスタム）
ウィンドウサイズを変えたい	**View Settings → Small / Medium / Large / Wide / Custom Size...**
サイドパネルを表示/非表示したい	**View Settings → Toggle Side Panel**

3.8 原子を選択したい

やりたいこと	操作
全原子を選択したい	**Selection → Select All** (Ctrl+A)
選択を解除したい	**Selection → Deselect All** (Escape)
特定の元素だけ選択したい（例: 全 C）	**Selection → Select by Element...**
特定の分子グループだけ選択したい	**Selection → Select by Molecule...**
ある原子の近傍を球状に選択したい	原子を選択 → **Selection → Select by Sphere...**
選択した分子だけにフォーカスしたい	**Selection → Focus on Selection** (Ctrl+F)。元に戻すには Ctrl+Shift+F
原子間の距離を測りたい	2 原子を選択するとステータスバーに距離が表示されます
結合角度を測りたい	3 原子を選択するとステータスバーに角度が表示されます
二面角を測りたい	4 原子を選択するとステータスバーに二面角が表示されます
測定履歴を見たい	**MoleculeTools → Measurement Log...**

3.9 操作を戻したい

やりたいこと	操作
直前の操作を元に戻したい	**MoleculeTools → Undo** (Ctrl+Z)。最大 50 ステップ
戻した操作をやり直したい	**MoleculeTools → Redo** (Ctrl+Y)

3.10 TOML 設定ファイルの書き方を知りたい

やりたいこと	操作
input.toml の全パラメータを知りたい	入出力ファイル形式 に全フィールドの説明があります
分子定義ファイル (data/*.toml) の書き方を知りたい	入出力ファイル形式 の「分子定義ファイル」セクションを参照
置換ルールの書き方を知りたい	入出力ファイル形式 の「置換設定」セクションを参照
実際の使用例を見たい	チュートリアル にベンゼン＋ナフタレンの混合結晶の例があります

第

FOUR

クイックスタート

このガイドでは、最短手順でパイプラインを実行し、結果を確認する方法を説明します。

4.1 パイプラインの実行

仮想環境をアクティベートした状態で、以下を実行します:

```
cd molecular_modeling
source .venv/bin/activate
python run.py
```

実行が完了すると、以下のような出力が表示されます:

```
=== Build system from TOML ===
...
=== Optimize Voronoi overlap ===
...
=== Write structure files ===
Written: ./structure_out/qe.in
Written: ./structure_out/POSCAR
Written: ./structure_out/INCAR
Written: ./structure_out/KPOINTS
Written: ./structure_out/structure.cif
```

4.2 入力ファイル

パイプラインは以下のファイルを読み込みます:

- `input/input.toml` — メインの設定ファイル（セルベクトル、分子リスト、繰り返し数など）
- `data/<name>.toml` — 分子定義ファイル（原子位置、格子ベクトルなど）

4.3 出力ファイル

`source/structure_out/` ディレクトリに以下のファイルが生成されます:

ファイル	説明
`qe.in`	Quantum ESPRESSO 入力ファイル
`POSCAR`	VASP 構造ファイル
`INCAR`	VASP 計算設定ファイル
`KPOINTS`	VASP k 点メッシュファイル
`structure.cif`	CIF 構造ファイル

`source/voronoi_pngs/` ディレクトリに Voronoi セルの可視化画像が生成されます。

4.4 GUI の起動

GUI を起動するには:

```
python molecular_modeling_gui.py
```

既存の CIF ファイルを開く場合:

```
python vis_cif.py structure.cif
```

GUI では以下の操作が可能です:

- 分子の追加・削除
- 元素の置換
- 構造ファイルのエクスポート
- 3D ビューの操作（回転・並進・ズーム）

詳細は *GUI 操作ガイド* を参照してください。

4.5 次のステップ

- チュートリアル — ベンゼン＋ナフタレンを使った詳細なチュートリアル
- *GUI 操作ガイド* — GUI の全操作ガイド
- コマンドラインツール — コマンドラインツールの使い方
- 入出力ファイル形式 — TOML 設定ファイルの全キー解説

- アーキテクチャ — 内部アーキテクチャの説明

第

FIVE

チュートリアル

このチュートリアルでは、ベンゼンとナフタレンの混合結晶を例に、パイプラインの全ステップを実行します。

5.1 ステップ 1: 分子定義の理解

まず、`data/benzene.toml` の内容を見てみましょう:

```toml
molecule_name = "benzene"

symbols = [
  "C", "C", "C", "C", "C", "C",
  "H", "H", "H", "H", "H", "H",
]

use_fractional = false

positions = [
  [ -1.113710, -0.097913, -0.811833 ],
  [  1.113710,  0.097913,  0.811833 ],
  # ... (12 原子分の座標)
]

lattice_vectors = [
  [ 5.522000,  0.000000,  0.000000],
  [ 0.000000,  5.439600,  0.000000],
  [-2.693272,  0.000000,  7.184363],
]

num_molecules = 2
```

(次のページに続く)

(前のページからの続き)

```
center_of_mass = [
  [ 0.000000,  0.000000,  0.000000],
  [-1.346636,  2.719800,  3.592182],
]
```

各フィールドの意味:

- `molecule_name`: 分子名（`input.toml` の `molecules` で参照）
- `symbols`: 元素記号のリスト。`"C6H6"` のようなコンパクト表記も可
- `use_fractional`: `true` の場合、座標を分率座標として解釈
- `positions`: 原子のデカルト座標 (Å)。`use_fractional = true` なら分率座標
- `lattice_vectors`: 格子ベクトル（行で a, b, c を定義）
- `num_molecules`: 単位胞中の分子数
- `center_of_mass`: 各分子の重心位置

各キーの詳細は 入出力ファイル形式 を参照してください。

ナフタレン（`data/naphthalene.toml`）も同様の構造です。元素記号はコンパクト表記 `"C10H8"` を使用しています。

5.2 ステップ 2: 入力ファイルの設定

`input/input.toml` でスーパーセルの構築方法を指定します:

```
# 乱数シード
rng = 1

# 格子ベクトル (Å)
cell = [
  [ 5.522000,  0.000000,  0.000000 ],
  [ 0.000000,  5.439600,  0.000000 ],
  [-3.501254,  0.000000,  9.339672 ],
]

# スーパーセルの繰り返し数 [a, b, c 方向]
supercell_repeat = [1, 1, 2]

# 使用する分子名のリスト
```

(次のページに続く)

(前のページからの続き)

```
molecules = ["benzene", "naphthalene"]

# 分率座標を使用するか
use_fractional = false

# 各分子の重心位置
center_of_mass = [
  [  0.000000,   0.000000,   0.000000 ],
  [ -1.750627,   2.719800,   4.669836 ],
]
```

`supercell_repeat = [1, 1, 2]` は c 軸方向に 2 倍のスーパーセルを作ることを意味します。2 種類の分子（ベンゼンとナフタレン）がそれぞれの重心位置に配置されます。

5.3 ステップ 3: パイプラインの実行

```
python run.py
```

パイプラインは以下の順序で処理を行います:

1. **Build**: TOML を読み込み、ASE `Atoms` オブジェクトを構築
2. **Group**: 原子を `mol_id` でグループ化
3. **Optimize**: Euler 角のグリッドサーチで Voronoi IoU を最大化
4. **Check**: Voronoi 体積のタイリング整合性を検証
5. **Render**: Voronoi セルオーバーレイの 3D 可視化
6. **Export**: QE, VASP, CIF ファイルを出力

5.4 ステップ 4: 出力ファイルの確認

5.4.1 構造ファイル

`source/structure_out/` に以下のファイルが生成されます:

qe.in (Quantum ESPRESSO):

```
&CONTROL
  calculation = 'scf'
  prefix = 'molecule'
```

(次のページに続く)

(前のページからの続き)

```
  outdir = './tmp'
/
&SYSTEM
  ecutwfc = 40
  ecutrho = 320
/
...
```

POSCAR (VASP): 原子位置が分率座標で記述されます。

structure.cif: 汎用 CIF フォーマットで出力されます。

5.5 ステップ 5: Voronoi 可視化の見方

`source/voronoi_pngs/` に生成される PNG 画像は、各サイトの Voronoi セルオーバーレイを示しています:

- **青い半透明ポリヘドロン**: システムの Voronoi セル（実際のセル分割）
- **赤い半透明ポリヘドロン**: 辞書の Voronoi セル（最適化ターゲット）
- IoU (Intersection over Union) 値が高いほど、分子配向の再現性が高い

5.6 ステップ 6: GUI でのインタラクティブ操作

GUI の全機能については *GUI 操作ガイド* を参照してください。

5.6.1 CIF ファイルの読み込み

```
python vis_cif.py source/structure_out/structure.cif
```

または GUI 内のメニューから:

MoleculeTools → Load CIF...

5.6.2 分子の追加

1. **MoleculeTools → Add Molecule...** を選択
2. ドロップダウンから分子名を選択（例: benzene）
3. 配置位置の座標を入力
4. **OK** をクリック

5.6.3 置換操作

GUI では 4 種類の置換操作が可能です:

1. **Global Element Substitution**: 全ての指定元素を一括置換
2. **Molecule Substitution**: 特定サイトの分子を別の分子に置換
3. **Molecule Atom Substitution**: 分子内の原子リストを変更
4. **Individual Atom Substitution**: 個別の原子を置換

5.6.4 エクスポート

Export → Export All (QE, VASP, CIF)... で全フォーマットに一括エクスポートできます。

5.7 ステップ 7: 置換機能の詳細

`input.toml` では 4 種類の置換テーブルを設定できます。

5.7.1 分子置換 (molecule_substitutions)

特定のセル位置にある分子を別の分子に置き換えます:

```
[[molecule_substitutions]]
cell_index       = [0, 0, 1]     # スーパーセル中のセル位置
molecule_index   = 1             # 分子インデックス (0-based)
new_molecule     = "benzene"     # 置換先の分子名
```

5.7.2 グローバル元素置換 (global_atom_substitutions)

全ての指定元素を一括で置き換えます:

```
[[global_atom_substitutions]]
old_symbol = "H"      # 置換元の元素
new_symbol = "He"     # 置換先の元素
```

5.7.3 分子内原子置換 (molecule_atom_substitutions)

特定の分子インスタンスの全原子を新しいリストで置き換えます:

```
[[molecule_atom_substitutions]]
cell_index     = [0, 0, 1]
molecule_index = 1
```

(次のページに続く)

(前のページからの続き)

```
new_symbols    = ["Si6O6"]    # コンパクト表記可
```

5.7.4 個別原子置換 (atom_substitutions)

1 つの原子だけを置き換えます:

```
[[atom_substitutions]]
cell_index     = [0, 0, 0]
molecule_index = 1
atom_index     = 2           # 分子内の原子インデックス (0-based)
new_symbol     = "Fe"
```

第

SIX

GUI 操作ガイド

Molecular Modeling GUI は ASE-GUI を拡張したグラフィカルインターフェースです。分子の追加、置換、エクスポートなどの操作をインタラクティブに行えます。

6.1 起動方法

```
# 空の構造で起動
python molecular_modeling_gui.py

# 既存の構造ファイルを指定して起動
python molecular_modeling_gui.py structure.cif

# CIF ビューアとして起動
python vis_cif.py structure.cif

# パイプラインから GUI を起動（edit_and_run.py）
python edit_and_run.py
```

6.2 ウィンドウ構成

GUI ウィンドウは以下の要素で構成されます:

1. **ファイルパネル（上部）**: アイコンボタン群（後述）
2. **3D ビューポート**: 分子構造の表示（回転・並進・ズーム可能）。左下に XYZ 座標軸インジケータ表示
3. **ツールバー**: Translate / Rotate / Zoom ボタン
4. **ボトムパネル**: Molecule Tools / Structure Tools のアイコンボタン群
5. **サイドパネル（右側）**: ビュー設定、Element Controls、Molecule Controls（折りたたみ可能）
6. **ステータスバー**: 選択中の原子情報

6.2.1 ウィンドウサイズの変更

View Settings メニューからウィンドウサイズを変更できます:

- **Small (450x450)**: コンパクト表示
- **Medium (700x700)**: 標準サイズ（デフォルト）
- **Large (1000x800)**: 大画面表示
- **Wide (1200x600)**: 横長表示
- **Custom Size...**: 任意のサイズを指定

ウィンドウの境界をドラッグしてサイズを変更することもできます。

6.2.2 ファイルパネル（上部）

ウィンドウ上部のアイコンボタンからよく使う操作にすばやくアクセスできます。

グループ	ボタン
Input	Load Structure（構造ファイル読み込み）/ Load Project（TOML プロジェクト読み込み）
Output	Save Project / Export Structure / Save Selection as Molecule / Screenshot
Edit	Undo / Redo
Pipeline	Run Optimize / Run All / Show Results / View PNGs

6.2.3 ボトムパネル

3D ビューポートの下側に配置されたアイコンボタン群です。

- **Molecule Tools**: Select（選択）/ Move（移動）/ Rotate（回転）/ Delete（削除）/ Drag（ドラッグモード）
- **Structure Tools**: Add Molecule / Substitution / Supercell / Cell Parameters

6.3 メニュー一覧

6.3.1 MoleculeTools メニュー

メニュー項目	説明
Undo (Ctrl+Z)	直前の操作を取り消す（最大 50 回）
Redo (Ctrl+Y)	取り消した操作をやり直す
Load Structure... (Ctrl+L)	構造ファイルを読み込み（CIF, XYZ, POSCAR, QE .in, PDB, GEN 等に対応）
Load CIF...	CIF ファイルを読み込み、構造を表示
Add Molecule... (Ctrl+J)	登録済み分子をシステムに追加
List Molecules/Groups	システム中の分子とグループを一覧表示
Save Selection as Molecule...	選択した原子群を新しい分子として TOML に保存
Global Element Substitution...	指定元素を全て別の元素に置換
Molecule Substitution...	特定サイトの分子を別の分子に置換
Molecule Atom Substitution...	分子内の原子リストを新しいリストに置換
Individual Atom Substitution...	1 つの原子を個別に置換
Batch Element Substitution...	複数の元素置換ルール（例: C→Si, H→F）を一括で定義・実行
Build Supercell...	繰り返し数を指定してスーパーセルを構築
Cell Parameters...	格子定数（a, b, c, α, β, γ）の表示・編集。格子ベクトルと体積も表示。Apply で原子座標をスケーリングして更新
Save Project... (Ctrl+K)	プロジェクト（input.toml + data/*.toml）を指定ディレクトリに保存
Load Project... (Ctrl+T)	プロジェクトディレクトリから構造を読み込み
Measurement Log...	測定ログ（距離・角度・二面角の履歴）を表示

6.3.2 Recent Files メニュー

最近開いた CIF ファイルが最大 10 件表示されます。ファイル名をクリックすると直接開きます。履歴は `~/.molecular_modeling_recent.json` に保存されます。

6.3.3 Export メニュー

メニュー項目	説明
Export All (QE, VASP, CIF)... (Ctrl+D)	全フォーマットに一括エクスポート
Quantum ESPRESSO...	QE 入力ファイル (.in) としてエクスポート
VASP...	POSCAR ファイルとしてエクスポート
CIF...	CIF ファイルとしてエクスポート

6.3.4 Selection メニュー

メニュー項目	説明
Select All (Ctrl+A)	表示中の全原子を選択
Deselect All (Escape)	全選択を解除
Select by Element...	元素記号を指定して原子を一括選択。リアルタイムプレビュー付き
Select by Molecule...	分子グループ（mol_id）を指定して原子を一括選択。カラースウォッチ付き
Select by Sphere...	選択中の原子を中心に、指定半径内の原子を選択。スライダーでリアルタイムプレビュー
Delete Selected Atoms (Delete)	選択した原子を削除（Undo 対応、分子グループメタデータも更新）
Delete Molecule...	選択した原子が属する分子グループ全体を削除
Move Molecule/Atoms...	選択した分子/原子を指定ベクトルで平行移動（分子全体/選択のみ切替可）
Rotate Molecule/Atoms...	選択した分子/原子を重心周りに Euler 角で回転（トラックボールウィジェット付き）
Focus on Selection (Ctrl+F)	フォーカスダイアログを表示（3D プレビュー付き）
Exit Focus Mode (Ctrl+Shift+F)	フォーカスモードを終了し、全原子の表示と元のビューを復元
Toggle Drag Mode	ドラッグモードの切り替え: 有効時、選択した原子をマウスドラッグで直接移動できる

6.3.5 Pipeline メニュー

メニュー項目	説明
Run Optimize (Voronoi) (Ctrl+P)	現在の構造に対して Voronoi IoU 最適化を実行。結果（IoU 値、Euler 角）がダイアログに表示される
Run Render (Voronoi PNGs)...	Voronoi セルオーバーレイの 3D 可視化 PNG を生成
Run Export (QE, VASP, CIF)...	構造ファイルを指定ディレクトリにエクスポート
Run All (Optimize → Render → Export)...	最適化 → 可視化 → エクスポートの全パイプラインを一括実行
Show Optimization Results... (Ctrl+H)	最適化結果（IoU 値、Euler 角、スケール）をテーブル形式で表示。CSV エクスポートおよび Voronoi PNG の閲覧も可能

注意: Pipeline メニューの各操作は `mol_id` 情報を持つ構造（TOML から構築した構造）に対してのみ実行できます。処理はバックグラウンドスレッドで実行されるため、GUI は操作中もフリーズしません。

6.3.6 View Settings メニュー

メニュー項目	説明
Atom Appearance...	原子の色・サイズを変更するパネルを表示。色モード（元素別/分子グループ別/タグ別）、元素ごとの色変更、原子サイズスケール、元素別半径の調整が可能
Color by Molecule Group	`mol_id` に基づき分子グループごとに異なる色で表示
Reset Colors	色設定を ASE デフォルト（jmol）に戻す
Show Bonds (Ctrl+B)	原子間のボンド（結合線）の表示/非表示を切り替え。共有結合半径に基づいて自動検出
Toggle Hydrogen	水素原子の表示/非表示を切り替え
Element Visibility...	元素ごとに表示/非表示を設定するダイアログを表示
Toggle Side Panel	サイドパネル（ビュー操作パネル）の表示/非表示を切り替え
Save Screenshot... (Ctrl+Shift+S)	現在の 3D ビューを PNG 画像として保存
Small / Medium / Large / Wide	ウィンドウサイズのプリセット変更
Custom Size...	任意のサイズを指定

6.3.7 Help メニュー

メニュー項目	説明
Keyboard Shortcuts...	キーボードショートカットとマウス操作の一覧をダイアログで表示
Open User Guide...	ビルド済み HTML マニュアルをブラウザで表示（`docs/_build/html/` が必要。未ビルドの場合はビルド手順を案内）
About...	バージョン情報を表示

6.4 ビュー操作ツールバー

3D ビューポートの下部にあるツールバーで、表示を操作できます。各ボタンは押し続けることで連続的に操作できます。

セクション	ボタン	説明
Translate	◀ ▶ ▲ ▼	ビューを上下左右に移動
Rotate	X- X+ Y- Y+ Z- Z+	X/Y/Z 軸周りに ±5° 回転
Zoom	- +	拡大・縮小
BG Color	White / Black / Dark / Light / Custom	背景色のプリセット変更またはカスタム色の選択
Actions	↩ ↪ ✕ ✦ ↻	Undo / Redo / 選択原子の削除 / 移動 / 回転（各ボタンにツールチップ付き）
Drag	[Drag]	ドラッグモード切り替え: 有効時、選択した原子をマウスドラッグで直接移動

6.4.1 ツールチップ

すべてのツールバーボタンにはツールチップ（マウスホバー時のフローティングヘルプ）が表示されます。ボタンの上にマウスカーソルを置くと、操作内容やキーボードショートカットが黄色のポップアップで表示されます。

6.4.2 サイドパネル

ウィンドウ右側にサイドパネルが表示されます。よく使うビュー設定にすばやくアクセスできます。パネルは **マウスホイール** で上下スクロールが可能です。内容がウィンドウに収まらない場合もスクロールして操作できます。

- **Focus / Unfocus**: フォーカスモードの開始/終了ボタン
- **Color Mode**: 色分けモードをラジオボタンで切り替え（元素別 / 分子グループ別 / タグ別）
- **Show Bonds**: ボンド（結合線）の表示/非表示チェックボックス (Ctrl+B)
- **3D Spheres**: 3D シェーディング球体レンダリングの有効/無効
- **Element Controls** *(折りたたみ可能)*: ヘッダーをクリックで展開/折りたたみ。元素ごとに [表示チェック][色スウォッチ][サイズスライダー] を一行で制御。全原子サイズスケールのスライダー（0.3〜3.0）も含む
- **Molecule Controls** *(折りたたみ可能)*: ヘッダーをクリックで展開/折りたたみ。分子グループごとに [表示チェック][色スウォッチ][選択ボタン] を一行で制御。**フォーカスモード中** はチェックボックスがフォーカス対象の追加/除去として機能
- **Atom List** *(折りたたみ可能、デフォルトは折りたたみ)*: ヘッダーをクリックで展開。全原子のインデックス (#)、元素、X/Y/Z 座標、分子グループ ID を表示するテーブル。行をクリックすると 3D ビューでその原子を選択
- **ステータス情報**: 原子数、選択数、元素内訳、グループ数をリアルタイム表示

サイドパネルは **View Settings → Toggle Side Panel** で表示/非表示を切り替えられます。

6.5 ステータスバー

画面下部のステータスバーには、選択状態に応じた情報が表示されます。

- **原子未選択時**: 全体情報 — 原子数、含有元素、分子グループ数、セルサイズ (例: `48 atoms | (C, H) | 4 groups | Cell: 5.5x5.4x18.7 Å`)
- **1 原子選択時**: 原子詳細 — インデックス、元素名、座標、所属分子名 (例: `#3 Carbon (C): 1.114, 0.098, 0.812 Å | mol_id=0 (benzene)`)
- **2 原子選択時**: 原子間距離（MIC 補正済み）、所属分子名 (例: `#0(C)–#6(H): 1.089 Å | benzene`)
- **3 原子選択時**: 結合角度（MIC 補正済み）、所属分子名 (例: `#0(C)–#1(C)–#2(C): 120.0° | benzene`)
- **4 原子選択時**: 二面角（MIC 補正済み）

測定結果は自動的にログに記録され、**MoleculeTools → Measurement Log...** で一覧を確認できます。

6.6 キーボードショートカット

ショートカット	操作
Ctrl+Z	元に戻す (Undo)
Ctrl+Y	やり直し (Redo)
Ctrl+A	表示中の全原子を選択
Escape	全選択を解除
Delete / Backspace	選択した原子を削除
Ctrl+L	構造ファイルを読み込み（CIF, XYZ, POSCAR 等）
Ctrl+J	分子を追加
Ctrl+D	全フォーマットに一括エクスポート
Ctrl+P	Voronoi IoU 最適化を実行
Ctrl+H	最適化結果を表示
Ctrl+K	プロジェクトを保存
Ctrl+T	プロジェクトを読み込み
Ctrl+B	ボンド表示の切り替え
Ctrl+F	選択原子にフォーカス（選択原子のみ表示）
Ctrl+Shift+F	フォーカスモードを終了（全原子を復元）
Ctrl+Shift+S	スクリーンショット保存

ASE-GUI 標準のキーボードショートカット（Ctrl+O で開く、Ctrl+S で保存など）もそのまま使用できます。

6.7 マウス操作

ASE-GUI 標準のマウス操作に加え、右クリックコンテキストメニューが使用できます:

- **左クリック**: 原子を選択
- **Shift + 左クリック**: 選択に追加
- **Ctrl + 左クリック**: 選択を反転
- **右クリック**: コンテキストメニュー表示（Select Molecule Group / Move / Rotate / Delete）
- **左ドラッグ**: ビューの回転
- **右ドラッグ** (または Shift + 右ドラッグ): ビューの並進
- **マウスホイール**: ズーム

6.7.1 座標軸インジケータ

3D ビューポートの左下に XYZ 座標軸インジケータが表示されます。現在のビュー回転に合わせて軸方向がリアルタイムに更新され、構造のどの方向を見ているか常に確認できます。各軸はカラーコード化されています: **X** = 赤、**Y** = 緑、**Z** = 青。

6.7.2 右クリックコンテキストメニュー

3D ビューポートで右クリックすると、以下の操作にすばやくアクセスできます:

- **Select Molecule Group**: クリック位置の原子が属する分子グループ全体を選択
- **Move...**: 移動ダイアログを表示
- **Rotate...**: 回転ダイアログを表示
- **Delete Selected**: 選択中の原子を削除
- **Delete Molecule**: 選択中の原子が属する分子を削除

6.7.3 スクリーンショット

Ctrl+Shift+S または ファイルパネルのカメラアイコンで、現在の 3D ビューを PNG ファイルとして保存できます。Pillow がインストールされている場合は PNG 形式、未インストールの場合は EPS 形式で保存されます。

6.8 ダイアログ詳細

6.8.1 Focus Selection ダイアログ

原子を選択した状態で **Focus on Selection** (Ctrl+F) を実行すると、プレビューウィンドウが表示されます。

ウィンドウは **非モーダル** で、メイン画面での原子クリックが可能です。

- **3D プレビュー** (上部): フォーカス対象の原子がカラー表示、非対象は薄いグレーで表示。マウスで回転・ズーム可能
- **ステータス**: フォーカス中の原子数を表示
- **Unit**:
  - **Atom**: 個別の原子単位で追加/除去
  - **Molecule**: クリックした原子が属する分子グループ全体で追加/除去
- **Molecules**: 各分子グループがチェックボックスと色スウォッチで表示される。チェックの ON/OFF でフォーカスセットに追加/除去

自動トグル動作:

メイン画面では、現在のフォーカス対象が選択状態（ハイライト）で表示されます。Add/Delete モードの切り替えは不要で、クリックするだけで自動的にトグルされます：

- フォーカス **外** の原子をクリック → フォーカスに **追加**
- フォーカス **中** の原子をクリック → フォーカスから **除去**

通常のクリック、Ctrl+ クリック、Shift+ クリックのいずれでも動作します。

操作例:

1. 原子を選択して Focus → 選択原子の所属分子がプレビューに表示され、メイン画面でフォーカス対象がハイライト
2. Unit を Molecule にして別の原子をクリック → その分子がプレビューに追加、メイン画面のハイライトも更新
3. Unit を Atom にしてフォーカス中の特定の原子をクリック → その原子だけ除去
4. ダイアログの Molecules チェックボックスでも分子単位で追加/除去が可能
5. OK でフォーカスモードに確定、Cancel で元の状態に復元

フォーカスモード中のサイドパネル操作:

フォーカスモード中は、サイドパネルの Molecule Controls のチェックボックスがフォーカス対象の追加/除去として機能します：

- チェックを入れる → その分子をフォーカスに追加して表示
- チェックを外す → その分子をフォーカスから除去して非表示

6.8.2 Add Molecule ダイアログ

- **Molecule**: ドロップダウンから `data/*.toml` に定義された分子を選択
- **Position (X, Y, Z)**: 配置位置をデカルト座標で入力
- **Add Mode**:
  - **Single**: 1 つの分子を指定位置に追加
  - **Unit Cell**: 分子定義の全分子を一括追加（格子ベクトルも設定）
- **3D Preview**: 選択した分子のプレビュー表示

6.8.3 Save Selection as Molecule ダイアログ

1. 3D ビューポートで原子を選択（Shift + クリックで複数選択）
2. **MoleculeTools → Save Selection as Molecule...** を選択
3. 分子名と単位胞中の分子数を入力
4. **OK** で `data/<name>.toml` として保存

6.8.4 Global Element Substitution ダイアログ

- **Old Symbol**: 置換元の元素記号（既存元素から選択）
- **New Symbol**: 置換先の元素記号を入力
- 全ての該当原子が一括で置換されます

6.8.5 Molecule Substitution ダイアログ

- **Molecule Index**: 置換対象の分子インデックス（原子を選択していれば自動設定）
- **New Molecule**: 置換先の分子名をドロップダウンから選択

6.8.6 Molecule Atom Substitution ダイアログ

特定の分子の全原子を新しい元素リストに置き換えます。

- **Molecule Index**: 対象の分子インデックス（原子選択時は自動設定）
- **New Symbols**: 新しい元素記号のリスト。コンパクト表記（`Si606` 等）も使用可能
- 原子数は変わらず、元素記号のみが変更されます

6.8.7 Individual Atom Substitution ダイアログ

分子内の特定の 1 原子だけを別の元素に置き換えます。

- **Molecule Index**: 対象の分子インデックス（原子選択時は自動設定）
- **Atom Index**: 分子内の原子インデックス（0 始まり）
- **New Symbol**: 置換先の元素記号

6.8.8 Batch Element Substitution ダイアログ

複数の元素置換ルールをまとめて定義・実行できます。

- 各行で「置換元の元素 → 置換先の元素」を指定
- **+** ボタンで行を追加、**x** ボタンで行を削除
- 例: C→Si と H→F を同時に実行可能
- **OK** で全ルールが一括適用されます

6.8.9 Build Supercell ダイアログ

現在の構造を繰り返し配置してスーパーセルを構築します。

- **a, b, c**: 各軸方向の繰り返し数を指定（整数）
- 格子ベクトルと原子座標が繰り返し数分だけ拡張されます

6.8.10 Move Molecule/Atoms ダイアログ

選択した原子や分子を平行移動します。

- **対象**:「分子グループ全体」または「選択原子のみ」を選択可能
- **移動量 (X, Y, Z)**: 各軸方向の移動量を Å 単位で入力
- **+/- ボタン**: 微調整ボタン（押し続けで連続操作可能）
- **Step**: 移動ステップ幅のプリセット（0.01 / 0.1 / 0.5 / 1.0 Å）
- **Apply**: 現在の移動量を適用、**Reset**: 初期位置に戻す

6.8.11 Rotate Molecule/Atoms ダイアログ

選択した原子や分子を重心周りに回転します。

- **対象**:「分子グループ全体」または「選択原子のみ」を選択可能
- **トラックボール**: マウスドラッグで直感的に回転操作

- **α (X), β (Y), γ (Z)**: Euler 角を度単位で入力
- **+/- ボタン**: 微調整ボタン（押し続けで連続操作可能）
- **Step**: 回転ステップ幅のプリセット（1 / 5 / 15 / 45 / 90°）
- **Apply**: 現在の回転を適用、**Reset**: 初期配向に戻す

6.8.12 Select by Element ダイアログ

元素ごとのチェックボックスで原子を選択します。

- 各元素にチェックボックスと原子数を表示
- チェックの変更で **リアルタイムに選択が更新** されます
- **モード**: Replace（既存の選択を置換）/ Add（既存の選択に追加）

6.8.13 Select by Molecule ダイアログ

分子グループ（mol_id）単位で原子を選択します。

- 各グループにチェックボックス、カラースウォッチ、原子数を表示
- **リアルタイムプレビュー** で選択状態が即座に反映されます
- **モード**: Replace / Add

6.8.14 Select by Sphere ダイアログ

選択中の原子を中心に、指定半径内の原子を一括選択します。

- **半径スライダー**: ドラッグで半径を調整（リアルタイムプレビュー）
- 選択範囲内の原子数と元素内訳を表示

6.8.15 Atom Appearance パネル

View Settings → Atom Appearance... から開くパネルです。3 つのタブがあります:

Color Mode タブ:

- **Default (by element)**: 元素ごとの標準色（jmol カラー）
- **By molecule group**: `mol_id` に基づき分子グループごとに色分け（20 色を循環使用）
- **By tag**: タグ値に応じた色分け
- カスタム配列がある場合、追加の色モードが自動的に表示されます

Element Colors タブ:

- 構造中の各元素について個別に色を変更可能
- **Change...** ボタンでカラーピッカーを表示
- **Reset to Default** で jmol デフォルトに戻す

Size タブ:

- **Atom Size Scale**: 全体の原子サイズスケール（0.1〜3.0、スライダーで調整）
- **Per-Element Radius Override**: 元素ごとの共有結合半径を個別に変更

6.8.16 Cell Parameters ダイアログ

MoleculeTools → Cell Parameters... から開きます。

- **Lattice lengths**: a, b, c（Å 単位）を表示・編集
- **Lattice angles**: α, β, γ（度単位）を表示・編集
- **Lattice vectors**: 現在の格子ベクトル（読み取り専用）
- **Volume**: 格子体積（Å³）
- **Apply**: 入力値で格子を更新（原子座標はスケーリング）

6.8.17 Export ダイアログ

- 出力先ディレクトリを選択
- 出力フォーマットを選択（QE / VASP / CIF / All）
- **OK** でエクスポート実行

第

SEVEN

コマンドラインツール

7.1 run.py — パイプライン実行

メインのパイプラインを実行します。

```
python run.py
```

処理フロー:

1. `input/input.toml` と `data/*.toml` からシステムを構築
2. 原子を分子グループにインデックス化
3. Voronoi IoU 最大化による分子配向の最適化
4. Voronoi 体積のタイリング整合性チェック
5. Voronoi セルオーバーレイの可視化 → `source/voronoi_pngs/`
6. 構造ファイルの出力 → `source/structure_out/`

入力ファイル（詳細は 入出力ファイル形式 を参照）:

- `input/input.toml` — 主設定
- `data/<name>.toml` — 分子定義

出力ファイル:

- `source/structure_out/qe.in` — Quantum ESPRESSO 入力
- `source/structure_out/POSCAR` — VASP 構造ファイル
- `source/structure_out/INCAR` — VASP 設定ファイル
- `source/structure_out/KPOINTS` — VASP k 点設定
- `source/structure_out/structure.cif` — CIF ファイル

- source/voronoi_pngs/*.png — Voronoi 可視化画像

7.2 molecular_modeling_gui.py — GUI 起動

ASE-GUI を拡張した分子モデリング GUI を起動します。詳細は *GUI 操作ガイド* を参照してください。

```
# 空の構造で起動
python molecular_modeling_gui.py

# 構造ファイルを指定して起動
python molecular_modeling_gui.py [structure_file]
```

引数:

引数	説明
structure_file (任意)	ASE が読み込み可能な構造ファイルのパス

ファイルを指定しない場合、20×20×20 Å のボックスが生成されます。

7.3 vis_cif.py — CIF ビューア

CIF ファイルを GUI で表示します。

```
# ファイルを指定して起動
python vis_cif.py <path_to_file.cif>

# ファイルダイアログで選択
python vis_cif.py
```

引数:

引数	説明
path_to_file.cif (任意)	CIF ファイルのパス。省略時はファイル選択ダイアログが表示される

備考: ASE の CIF パーサーで読み込みに失敗した場合、_citation* タグを除去して再試行するフォールバック機能があります。

7.4 edit_and_run.py — GUI 編集＋最適化

GUI での手動編集後に Voronoi 最適化を実行するパイプラインです。

```
python edit_and_run.py
```

処理フロー:

1. TOML からシステムを構築
2. **GUI を起動**（ユーザーが構造を編集）
3. GUI を閉じると、編集後のシステムで処理を続行
4. 分子グループの構築
5. Voronoi IoU 最適化
6. 可視化とエクスポート

7.5 テストの実行

```
# 全テスト実行
pytest

# 特定のテストファイル
pytest tests/test_voronoi_geom.py

# 特定のテストケース
pytest tests/test_voronoi_geom.py::TestPolyhedron::test_volume

# 遅いテストをスキップ
pytest -m "not slow"
```

第

EIGHT

入出力ファイル形式

8.1 ファイル構成の概要

本ツールキットは 2 種類の TOML ファイルを使用します:

1. **入力設定** (`input/input.toml`): スーパーセルの格子・繰り返し数・分子配置・置換ルールを定義
2. **分子定義** (`data/<name>.toml`): 各分子の原子構造・格子ベクトル・重心を定義

これらのファイルの関係を以下に示します:

```
input/input.toml
├── molecules = ["benzene", "naphthalene"]
│       ↓                      ↓
│   data/benzene.toml    data/naphthalene.toml
│   (原子座標・格子)      (原子座標・格子)
│
├── cell = [...]            ← スーパーセルの格子ベクトル
├── supercell_repeat = [1,1,2]  ← 繰り返し数
├── center_of_mass = [...]      ← 各分子の配置位置
└── [[substitutions]]           ← 置換ルール（任意）
```

処理の流れ:

1. `input.toml` を読み込み、格子ベクトルと繰り返し数を取得
2. `molecules` リストに基づき `data/<name>.toml` を読み込み
3. `supercell_repeat` で指定された繰り返し分のセルを生成
4. 各セルに `center_of_mass` の位置で分子を配置
5. 置換ルールを適用（分子置換 → グローバル元素置換 → 分子内原子置換 → 個別原子置換）
6. ASE `Atoms` オブジェクトを構築

8.2 入力設定: input/input.toml

パイプラインの動作を制御するメインの設定ファイルです。

8.2.1 基本設定

```
# 乱数シード（整数）
# optimize, render の個別シードが未設定の場合にフォールバック
rng = 1

# 格子ベクトル (3×3 配列, 単位: Å)
# 行で a, b, c ベクトルを定義
cell = [
  [ 5.522000,  0.000000,  0.000000 ],   # a−ベクトル
  [ 0.000000,  5.439600,  0.000000 ],   # b−ベクトル
  [−3.501254,  0.000000,  9.339672 ],   # c−ベクトル
]

# スーパーセル繰り返し数 [a 方向, b 方向, c 方向]
supercell_repeat = [1, 1, 2]

# 使用する分子名のリスト（data/<name>.toml と対応）
molecules = ["benzene", "naphthalene"]

# 分率座標を使用するか (true: 分率座標, false: デカルト座標)
use_fractional = false

# 各分子の重心位置（molecules と同じ順序）
center_of_mass = [
  [  0.000000,   0.000000,   0.000000 ],
  [ −1.750627,   2.719800,   4.669836 ],
]
```

8.2.2 全キー一覧

キー	型	説明
rng	int	グローバル乱数シード。optimize_rng / render_rng 未設定時のフォールバック
optimize_rng	int (任意)	Voronoi 最適化ステージ専用の乱数シード。省略時は rng を使用
render_rng	int (任意)	可視化ステージ専用の乱数シード。省略時は rng を使用
cell	array[3][3]	格子ベクトル（行で a, b, c, 単位 Å）
supercell_repeat	array[3]	スーパーセル繰り返し数 [nx, ny, nz]
molecules	array[str]	使用する分子名（data/<name>.toml に対応）
use_fractional	bool	true の場合 center_of_mass を分率座標として解釈
center_of_mass	array[N][3]	各分子の重心座標（molecules と同じ数）

8.2.3 cell_index とスーパーセルの関係

supercell_repeat = [nx, ny, nz] でスーパーセルを構築すると、各セルに [ix, iy, iz] のインデックスが割り当てられます。

- ix: 0 から nx-1
- iy: 0 から ny-1
- iz: 0 から nz-1

例えば supercell_repeat = [1, 1, 2] の場合:

```
セル [0, 0, 0] — 元の単位胞
セル [0, 0, 1] — c 軸方向に 1 つずらしたコピー
```

各セル内に molecules で定義した分子が配置されるため、molecule_index = 0 はベンゼン、molecule_index = 1 はナフタレンを指します。

置換テーブルの cell_index は、このスーパーセル中のセル位置を指定します。

8.2.4 置換テーブル

4 種類の置換テーブルを [[テーブル名]] 形式で複数定義できます。置換は以下の順序で適用されます:

1. **分子置換** (molecule_substitutions): 分子全体を別の分子に入れ替え
2. **グローバル元素置換** (global_atom_substitutions): 全元素を一括置換
3. **分子内原子置換** (molecule_atom_substitutions): 特定分子の原子リストを変更
4. **個別原子置換** (atom_substitutions): 1 原子だけ変更

molecule_substitutions（分子置換）

特定のセル位置にある分子を別の分子に置き換えます。

```
[[molecule_substitutions]]
cell_index       = [0, 0, 1]        # スーパーセル中のセル座標
molecule_index   = 1                # 分子インデックス (0-based)
new_molecule     = "benzene"        # 置換先の分子名
```

キー	型	説明
cell_index	array[3]	スーパーセル中のセル位置 [ix, iy, iz]
molecule_index	int	セル内の分子インデックス (0-based)
new_molecule	str	置換先の分子名

global_atom_substitutions（グローバル元素置換）

全てのシステムで指定元素を一括置換します。

```
[[global_atom_substitutions]]
old_symbol = "H"       # 置換元の元素記号
new_symbol = "He"      # 置換先の元素記号
```

キー	型	説明
old_symbol	str	置換元の元素記号
new_symbol	str	置換先の元素記号

molecule_atom_substitutions（分子内原子置換）

特定の分子インスタンスの原子リストを新しいリストに置き換えます。

注釈

この置換は原子リスト全体を上書きするため、同じ分子に対するグローバル元素置換（global_atom_substitutions）の結果は無効になります。

```
[[molecule_atom_substitutions]]
cell_index     = [0, 0, 1]
molecule_index = 1
new_symbols    = ["Si6O6"]     # コンパクト表記可
```

キー	型	説明
cell_index	array[3]	スーパーセル中のセル位置
molecule_index	int	セル内の分子インデックス
new_symbols	array[str]	新しい元素記号リスト（コンパクト表記可）

atom_substitutions（個別原子置換）

特定の 1 原子だけを置き換えます。

```
[[atom_substitutions]]
cell_index     = [0, 0, 0]
molecule_index = 1
atom_index     = 2            # 分子内の原子インデックス (0-based)
new_symbol     = "Fe"
```

キー	型	説明
cell_index	array[3]	スーパーセル中のセル位置
molecule_index	int	セル内の分子インデックス
atom_index	int	分子内の原子インデックス (0-based)
new_symbol	str	置換先の元素記号

8.3 分子定義: data/<name>.toml

各分子の構造情報を定義するファイルです。`input.toml` の `molecules` リストに記載された名前とファイル名（`.toml` 拡張子を除く）が対応します。

注釈

`data/*.toml` の `center_of_mass` は **分子辞書の参照情報** として使用されます。スーパーセル中での分子配置位置は `input.toml` の `center_of_mass` で決定されます。

`data/*.toml` の `lattice_vectors` は **分子固有の格子** です。スーパーセル全体の格子は `input.toml` の `cell` で定義されます。混合結晶（異なる分子の共結晶）の場合、これらは異なる値になります。

8.3.1 基本構造

```
molecule_name = "benzene"

# 元素記号: 個別リストまたはコンパクト表記
symbols = ["C", "C", "C", "C", "C", "C", "H", "H", "H", "H", "H", "H"]
# または: symbols = ["C6H6"]

# 座標系の指定
use_fractional = false

# 原子座標 (N×3 配列)
positions = [
  [-1.113710, -0.097913, -0.811833],
  [ 1.113710,  0.097913,  0.811833],
  # ...
]

# 格子ベクトル (3×3 配列, 単位: Å)
lattice_vectors = [
  [ 5.522000,  0.000000,  0.000000],
  [ 0.000000,  5.439600,  0.000000],
  [-2.693272,  0.000000,  7.184363],
]

# 単位胞中の分子数
num_molecules = 2

# 各分子の重心位置
center_of_mass = [
  [ 0.000000,  0.000000,  0.000000],
  [-1.346636,  2.719800,  3.592182],
```

(次のページに続く)

(前のページからの続き)

```
]

# Voronoi 参照分子インデックス (省略時: 0)
# voronoi_ref_id = 0
```

8.3.2 全キー一覧

キー	型	必須	説明
molecule_name	str	Yes	分子の識別名
symbols	array[str]	Yes	元素記号のリスト。コンパクト表記可（例: "C6H6"）
use_fractional	bool	No	true の場合、positions と center_of_mass を分率座標として解釈
positions	array[N][3]	Yes	原子の座標（デカルト Å または分率座標）
lattice_vectors	array[3][3]	Yes	格子ベクトル（行で a, b, c）
num_molecules	int	Yes	単位胞中の分子数
center_of_mass	array[K][3]	Yes	K 個の分子の重心座標
voronoi_ref_id	int	No	Voronoi セル計算の参照分子インデックス (デフォルト: 0)

8.3.3 コンパクト表記

symbols フィールドでは、元素記号と数を連結した表記が使えます:

```
"C6H6"    → ["C", "C", "C", "C", "C", "C", "H", "H", "H", "H", "H", "H"]
"C10H8"   → ["C"] × 10 + ["H"] × 8
"Si6O6"   → ["Si"] × 6 + ["O"] × 6
```

8.3.4 座標系の変換

use_fractional = true の場合、内部で以下の変換が行われます:

```
pos_cart = pos_frac @ lattice_vectors
pos_frac = pos_cart @ inv(lattice_vectors)
```

注釈

`input.toml` と `data/*.toml` の `use_fractional` は **独立** です。一方が分率座標を使い、もう一方がデカルト座標を使うことも可能です。

8.4 完全な設定例

以下に、同梱のベンゼン・ナフタレン混合結晶の完全な設定例を示します。

8.4.1 input/input.toml の完全例

```toml
# 乱数シード
rng = 1

# スーパーセルの格子ベクトル（Å）
# ベンゼンとナフタレンの混合結晶の格子
cell = [
  [ 5.522000,  0.000000,  0.000000 ],
  [ 0.000000,  5.439600,  0.000000 ],
  [-3.501254,  0.000000,  9.339672 ],
]

# c 軸方向に 2 倍のスーパーセルを構築
supercell_repeat = [1, 1, 2]

# 2 種類の分子を使用
# data/benzene.toml と data/naphthalene.toml を参照
molecules = ["benzene", "naphthalene"]

# デカルト座標を使用
use_fractional = false

# 各分子の配置位置（molecules と同じ順序）
# benzene → 原点, naphthalene → オフセット位置
center_of_mass = [
  [  0.000000,   0.000000,   0.000000 ],
  [ -1.750627,   2.719800,   4.669836 ],
]

# --- 以下はオプションの置換ルール ---
```

(次のページに続く)

(前のページからの続き)

```
# セル [0,0,1] の 2 番目の分子 (naphthalene) を benzene に置換
[[molecule_substitutions]]
cell_index       = [0, 0, 1]
molecule_index   = 1
new_molecule     = "benzene"

# 全ての H を He に置換
[[global_atom_substitutions]]
old_symbol = "H"
new_symbol = "He"

# セル [0,0,1] の 2 番目の分子の原子リストを Si6O6 に変更
[[molecule_atom_substitutions]]
cell_index     = [0, 0, 1]
molecule_index = 1
new_symbols    = ["Si6O6"]

# セル [0,0,0] の 2 番目の分子の 3 番目の原子を Fe に変更
[[atom_substitutions]]
cell_index     = [0, 0, 0]
molecule_index = 1
atom_index     = 2
new_symbol     = "Fe"
```

8.4.2 data/benzene.toml の完全例

元素記号を **個別リスト** で記述する形式です。

```
molecule_name = "benzene"

# 12 原子を個別に列挙
symbols = [
  "C", "C", "C", "C", "C", "C",
  "H", "H", "H", "H", "H", "H",
]

use_fractional = false
```

(次のページに続く)

(前のページからの続き)

```
# 第 1 分子の原子座標 (Å)
# symbols と同じ順序で対応
positions = [
  # 炭素原子 (6 個)
  [ -1.113710, -0.097913, -0.811833 ],
  [  1.113710,  0.097913,  0.811833 ],
  [ -0.115564, -1.087920, -0.865716 ],
  [  0.115564,  1.087920,  0.865716 ],
  [ -0.979384,  0.951930,  0.052446 ],
  [  0.979384, -0.951930, -0.052446 ],
  # 水素原子 (6 個)
  [ -1.861055, -0.158292, -1.362155 ],
  [  1.861055,  0.158292,  1.362155 ],
  [ -0.197159, -1.816826, -1.437591 ],
  [  0.197159,  1.816826,  1.437591 ],
  [ -1.651612,  1.593259,  0.089805 ],
  [  1.651612, -1.593259, -0.089805 ],
]

# ベンゼン結晶の格子ベクトル
lattice_vectors = [
  [ 5.522000,  0.000000,  0.000000],
  [ 0.000000,  5.439600,  0.000000],
  [-2.693272,  0.000000,  7.184363],
]

# 単位胞中に 2 分子
num_molecules = 2

# 各分子の重心位置
center_of_mass = [
  [ 0.000000,  0.000000,  0.000000],
  [-1.346636,  2.719800,  3.592182],
]
```

8.4.3 data/naphthalene.toml の完全例

元素記号を **コンパクト表記** で記述する形式です。

```
molecule_name = "naphthalene"

# コンパクト表記: C10H8 → C×10 + H×8 = 18 原子
symbols = ["C10H8"]

use_fractional = false

# 第 1 分子の原子座標 (Å)
# コンパクト表記の展開順 (C が先、H が後) と対応
positions = [
  # 炭素原子 (10 個)
  [ -0.873883,  0.110730,  2.365899 ],
  [  0.873883, -0.110730, -2.365899 ],
  [ -0.125794,  0.966533,  1.590346 ],
  [  0.125794, -0.966533, -1.590346 ],
  [  0.213579,  0.617262,  0.251858 ],
  [ -0.213579, -0.617262, -0.251858 ],
  [  0.981797,  1.483077, -0.575778 ],
  [ -0.981797, -1.483077,  0.575778 ],
  [ -1.309714, -1.131450,  1.848626 ],
  [  1.309714,  1.131450, -1.848626 ],
  # 水素原子 (8 個)
  [ -1.094109,  0.351038,  3.237058 ],
  [  1.094109, -0.351038, -3.237058 ],
  [  0.159973,  1.778750,  1.942805 ],
  [ -0.159973, -1.778750, -1.942805 ],
  [  1.263671,  2.303540, -0.240443 ],
  [ -1.263671, -2.303540,  0.240443 ],
  [ -1.818179, -1.705126,  2.375888 ],
  [  1.818179,  1.705126, -2.375888 ],
]

# ナフタレン結晶の格子ベクトル（ベンゼンとは異なる）
lattice_vectors = [
  [ 8.034800,  0.000000,  0.000000],
  [ 0.000000,  5.889900,  0.000000],
  [-4.738553,  0.000000,  7.134798],
```

(次のページに続く)

(前のページからの続き)

```
]

# 単位胞中に 2 分子
num_molecules = 2

# 各分子の重心位置
center_of_mass = [
  [ 0.000000,  0.000000,  0.000000],
  [-1.346636,  2.719800,  3.592182],
]
```

8.4.4 新しい分子定義の作成方法

独自の分子を追加するには、以下の手順で `data/<name>.toml` を作成します:

1. `molecule_name` に分子名を設定
2. `symbols` で元素記号を定義（個別リストまたはコンパクト表記）
3. `positions` に **第 1 分子** の原子座標を記述（重心が原点付近になるように）
4. `lattice_vectors` にその分子の結晶格子を設定
5. `num_molecules` に単位胞あたりの分子数を設定
6. `center_of_mass` に各分子の重心位置を記述

Tip

GUI の **MoleculeTools → Save Selection as Molecule...** を使うと、3D ビューで選択した原子群を `data/*.toml` として保存できます。

付録

A

VORONOI 最適化の詳細

本ツールキットの中核機能である Voronoi セル IoU 最適化について、アルゴリズムの詳細を解説します。

A.1 問題設定

有機分子結晶や分子性固体のモデリングでは、単位胞中に配置された分子の**配向**（向き）を適切に決定する必要があります。本ツールキットでは、分子の重心を Voronoi 分割の母点として、系の Voronoi セルと辞書（参照結晶）の Voronoi セルの **幾何学的重なり**を最大化することで最適な配向を自動的に決定します。

ここでの「系（system）」とは実際に構築したスーパーセル、「辞書（dictionary）」とは参照となる単結晶構造を指します。

A.2 Voronoi セルとは

結晶中の各分子サイトの重心を母点として Voronoi 分割を行うと、空間が各サイトに「最も近い」領域に分割されます。この領域が Voronoi セル（ボロノイ多面体）です。

```
Voronoi セルの性質:
- 各母点に対して 1 つの凸多面体が対応
- 全セルの体積の合計 = 単位胞体積（空間を隙間なく充填）
- セルの形状は分子間の相対位置で決まる
```

本ツールキットでは、周期境界条件を考慮した Voronoi 分割を行います。具体的には、±1 セル分のイメージ（27 個の周期イメージ）を生成してから SciPy の半空間交差法（HalfspaceIntersection）で Voronoi セルの頂点を計算しています。

A.3 IoU（Intersection over Union）

IoU は 2 つの立体の重なり度合いを表す指標で、0 から 1 の値をとります。

```
                交差体積（Overlap）
IoU = ──────────────────────────────────────────────────────
       V_sys + V_dict − 交差体積

IoU = 1: 2 つの Voronoi セルが完全に一致
IoU = 0: 2 つの Voronoi セルが全く重ならない
```

ここで:

- **V_sys**: 系（スーパーセル）の Voronoi セル体積
- **V_dict**: 辞書（参照結晶）の Voronoi セル体積（スケーリング後）
- **交差体積**: 2 つのセルが重なる部分の体積

交差体積の計算には、系と辞書の両方の母点を統合した Voronoi 分割を行い、参照点に対する統合セルの体積を求めます。これは V_sys と V_dict の交差体積に相当します。

A.4 サイトごとの最適化手順

最適化は各分子サイトについて独立に行われます。1 サイトあたりの処理は以下の 5 ステップです。

A.4.1 ステップ 1: 系の Voronoi セル体積の計算

スーパーセルの全分子重心を母点とし、対象サイトの重心における Voronoi セル体積 V_sys を計算します。

```
入力: スーパーセル格子 A_sys, 全分子の重心座標
出力: V_sys (Å³)
```

系の Voronoi セルは最適化を通じて変化しません（配向を変えても重心位置は変わらないため）。

A.4.2 ステップ 2: 辞書の Voronoi セル体積の計算

辞書（参照結晶）の格子ベクトルと重心を使い、辞書側の Voronoi セル体積 V_dict を計算します。

```
入力: 辞書格子 A_dict, 辞書の重心座標
出力: V_dict (Å³)
```

A.4.3 ステップ 3: 体積スケーリング

系と辞書の Voronoi セル体積が異なる場合（混合結晶など）、辞書の格子と重心に均一スケーリングを適用して体積を合わせます。

```
スケール係数: s = (V_sys / V_dict)^(1/3)
```

このスケーリングにより、辞書の Voronoi セルの体積が系のそれと同じになります。立方根を使うのは、3 次元の均一スケーリングでは体積が s^3 倍になるためです。

A.4.4 ステップ 4: Euler 角の最適化

辞書の Voronoi セルを回転させ、系の Voronoi セルとの IoU を最大化する Euler 角 (α, β, γ) を探索します。

粗いグリッドサーチ（Phase 1）:

```
α, β, γ それぞれを [−180°, 180°] の範囲で 13 分割
→ 13 × 13 × 13 = 2,197 通りの組み合わせを評価
→ 最も IoU が高い角度を初期解として記録
```

Nelder-Mead 精密化（Phase 2）:

```
グリッドサーチの最良解を初期値として Nelder–Mead 法で精密化
– 目的関数: f(α, β, γ) = 1 − IoU（最小化）
– 角度許容誤差: 0.001°
– IoU 許容誤差: 0.001
– 最大反復回数: 2,000
– 最大関数評価回数: 10,000
```

Nelder-Mead 法は微分を使わない直接探索法であり、IoU のような微分が困難な目的関数に適しています。

Euler 角の回転規則:

```
回転順序: XYZ（外因性）
– α: X 軸周りの回転
– β: Y 軸周りの回転
– γ: Z 軸周りの回転

回転行列 R = R_z(γ) × R_y(β) × R_x(α)
```

A.4.5 ステップ 5: 原子座標への適用

最適化された回転角とスケールを、対象サイトの全原子の座標に適用します。

```
各原子の新座標:
  pos_new = p_com + s × R × (pos_old − p_com)
```

(次のページに続く)

(前のページからの続き)

```
p_com: サイトの重心
s:     スケール係数（ステップ 3）
R:     最適回転行列（ステップ 4）
```

回転とスケーリングは重心を中心に行われるため、分子の重心位置は変わりません。

A.5 辞書の Voronoi セルの構築方法

辞書の Voronoi セルは以下の手順で構築されます:

1. 辞書の重心から参照分子（`voronoi_ref_id`、デフォルト 0）を原点に移動
2. Euler 角 (α, β, γ) で回転
3. スケール s で均一拡縮
4. 系の対象サイトの重心位置に平行移動
5. 辞書格子も同じ回転・スケールを適用し、±1 セル分のイメージを生成
6. 系と辞書の全母点から統合 Voronoi 分割を実行

この手順により、辞書の Voronoi セルが系の座標空間に配置され、IoU の計算が可能になります。

A.6 体積タイリング検証

最適化後に、Voronoi セルの体積合計がスーパーセル体積と一致するか検証します。

```
検証条件:
  |Σ V_i − V_cell| ⊠ abs_tol  または  |Σ V_i − V_cell| / V_cell ⊠ rel_tol

デフォルト許容値:
  abs_tol = 1e-6 Å³
  rel_tol = 1e-3 (0.1%)
```

この検証に通らない場合、Voronoi 分割に問題がある可能性があります（分子の配置が密すぎる、格子が極端に歪んでいるなど）。

A.7 最適化結果の解釈

最適化の結果は `system.info` に格納され、GUI の **Pipeline → Show Optimization Results** で確認できます。

出力項目	意味
IoU	系と辞書の Voronoi セルの重なり度合い（0〜1）。1 に近いほど、辞書の分子配向が系で良く再現されている
α, β, γ	最適化された Euler 角（度）。辞書の分子配向に対する回転量
scale	体積スケーリング係数。1.0 なら系と辞書の Voronoi 体積が同じ。1 より大きい場合、系のサイトが辞書より広い

IoU の目安:

- **0.9 以上**: 辞書の配向が系で非常に良く再現されている
- **0.7〜0.9**: おおむね良い再現
- **0.7 未満**: 系と辞書の環境が大きく異なる（混合結晶での異種分子の影響など）

A.8 可視化（Voronoi オーバーレイ PNG）

最適化結果は 3D 画像として可視化されます:

- **青い半透明多面体**: 系の Voronoi セル（実際の空間分割）
- **赤い半透明多面体**: 辞書の Voronoi セル（最適化で回転・スケールされた参照セル）
- **オレンジの半透明多面体**: 2 つのセルの交差部分

IoU が高いほど、青と赤の多面体がよく重なり、ほぼ同じ形状になります。

A.9 計算に使用するライブラリ

ライブラリ	用途
SciPy (`scipy.spatial`)	Voronoi 分割、半空間交差法、凸包計算、Nelder-Mead 最適化
SciPy (`scipy.spatial.transform`)	Euler 角による回転行列の生成
trimesh	多面体メッシュの構築、体積計算、交差計算
manifold3d	ポリヘドラのブーリアン演算（交差・和集合）
NumPy	数値配列演算、線形代数

付録

B

アーキテクチャ

B.1 パイプラインフロー

メインパイプライン（ run.py → source/main.py ）は以下の順序で処理を実行します:

```
input/input.toml + data/*.toml
        |
        ▼
┌──────────────────────────────────────────┐
|  build_system_from_toml  |  TOML → ASE Atoms 構築
|                          |  スーパーセル生成、置換適用
└──────────────┬───────────┴──────────────┘
           |  system (Atoms + system.info)
           ▼
┌──────────────────────────────────────────┐
|  build_molecule_groups   |  mol_id による原子グループ化
└──────────────┬───────────┴──────────────┘
           |  groups: list[list[int]]
           ▼
┌──────────────────────────────────────────┐
| optimize_voronoi_overlap |  Euler 角グリッドサーチ +
|                          |  Nelder-Mead 最適化
└──────────────┬───────────┴──────────────┘
           |  optimized_angles_deg, scales, iou
           ▼
┌──────────────────────────────────────────────────────┐
| check_system_voronoi_volume      |  Voronoi 体積の
| _tiling (voronoi_geom.py)        |  タイリング整合性検証
└──────────────┬───────────────────┴──────────────────┘
           |
```

(次のページに続く)

(前のページからの続き)

```
          ▼
┌──────────────────────────────────────────┐
│ render_voronoi_overlays │  3D matplotlib 可視化
└──────────────────┬───────────────────────┘
           │  voronoi_pngs/*.png
           ▼
┌──────────────────────────────────────────┐
│     write_structure     │  QE / VASP / CIF エクスポート
└──────────────────┬───────────────────────┘
           │
           ▼
structure_out/  (qe.in, POSCAR, INCAR, KPOINTS, structure.cif)
```

B.2 各モジュールの役割

B.2.1 build_system_from_toml.py

入力: `input/input.toml`, `data/*.toml`

出力: ASE `Atoms` オブジェクト

主な処理:

1. TOML ファイルの読み込みとパース
2. 分率座標 → デカルト座標への変換（`use_fractional` が true の場合）
3. スーパーセルの構築（繰り返しタイリング）
4. 各分子の配置（重心位置への並進）
5. 4 種類の置換テーブルの適用
6. `mol_id` 配列の生成（各原子がどのサイトに属するか）
7. `system.info` へのメタデータ設定

B.2.2 build_molecule_groups.py

入力: `Atoms` オブジェクト（`mol_id` 配列付き）

出力: `list[list[int]]` — サイトごとの原子インデックスリスト

`mol_id` 配列を走査して、同じ値を持つ原子のインデックスをグループ化します。

B.2.3 voronoi_geom.py

コアのジオメトリモジュールです。以下を提供します:

- **Polyhedron クラス**: 凸多面体の表現（頂点 + 面）
- **Voronoi セル計算**: SciPy の halfspace intersection による周期的 Voronoi 分割
- **IoU 計算**: trimesh と manifold3d を使った体積ベースの Intersection over Union
- **回転・スケーリング**: Euler 角による多面体の回転と均一スケーリング
- **周期境界処理**: セル境界でのラッピング

B.2.4 optimize_voronoi_overlap.py

処理: サイトごとに Voronoi IoU を最大化する Euler 角を探索

アルゴリズム:

1. システム Voronoi セル体積 `V_sys` を計算
2. 辞書 Voronoi セル体積 `V_dict` を計算
3. 均一スケール `s = (V_sys / V_dict)^(1/3)` を設定
4. Euler 角 (α, β, γ) の粗いグリッドサーチ（デフォルト 13×13×13）
5. 最良点から Nelder-Mead 法で精密最適化
6. 最適な回転とスケーリングを原子位置に適用

結果は `system.info` に格納:

- `optimized_angles_deg`: 最適 Euler 角 (N×3)
- `optimized_scales`: 最適スケール (N,)
- `optimized_iou`: IoU 値 (N,)

B.2.5 render_voronoi_overlays.py

matplotlib の 3D プロットでシステムと辞書の Voronoi セルを重ね合わせて可視化します。

- 青い半透明: システム Voronoi セル
- 赤い半透明: 辞書 Voronoi セル
- サイトごとに 1 枚の PNG を生成

B.2.6 write_structure.py

ASE の I/O 機能を使って、以下のフォーマットで構造ファイルを出力します:

- **Quantum ESPRESSO**: `qe.in` (SCF 計算設定付き)
- **VASP**: `POSCAR`, `INCAR`, `KPOINTS`
- **CIF**: `structure.cif`

B.3 system.info メタデータ

パイプラインの各ステージ間でデータを受け渡すために、ASE `Atoms` オブジェクトの `info` 辞書を使用します。

キー	型	説明
`groups`	list[list[int]]	サイトごとの原子インデックス
`site_mol_name`	list[str]	サイトごとの分子名
`dict_meta`	dict	分子ごとのメタデータ（格子ベクトル、重心、voronoi_ref_id）
`rng`	dict	乱数シード（global, optimize, render）
`optimized_angles_deg`	ndarray(N,3)	最適化後の Euler 角（度）
`optimized_scales`	ndarray(N,)	最適化後のスケール係数
`optimized_iou`	ndarray(N,)	最適化後の IoU 値

B.3.1 原子配列

配列名	型	説明
`mol_id`	int[N_atoms]	各原子が属するサイトのインデックス

B.4 GUI アーキテクチャ

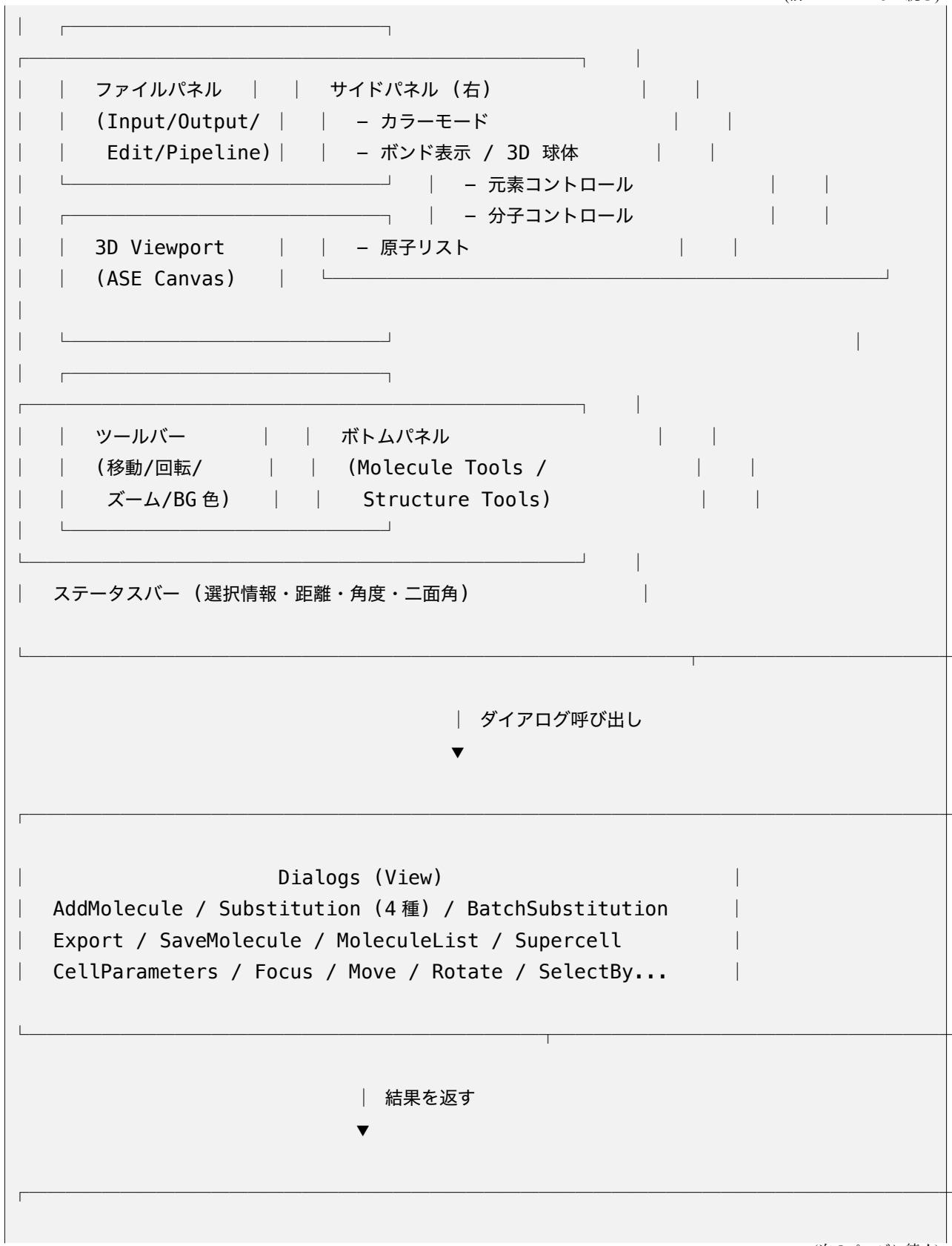
GUI は MVC (Model-View-Controller) パターンに近い設計です。

```
┌──────────────────────────────────────────────────────────────────────────────

│              MolecularModelingGUI (View)                    │
│                                                               │
│  メニューバー: MoleculeTools / Recent Files / Export /      │
│              Selection / Pipeline / View Settings / Help    │
│                                                               │
```

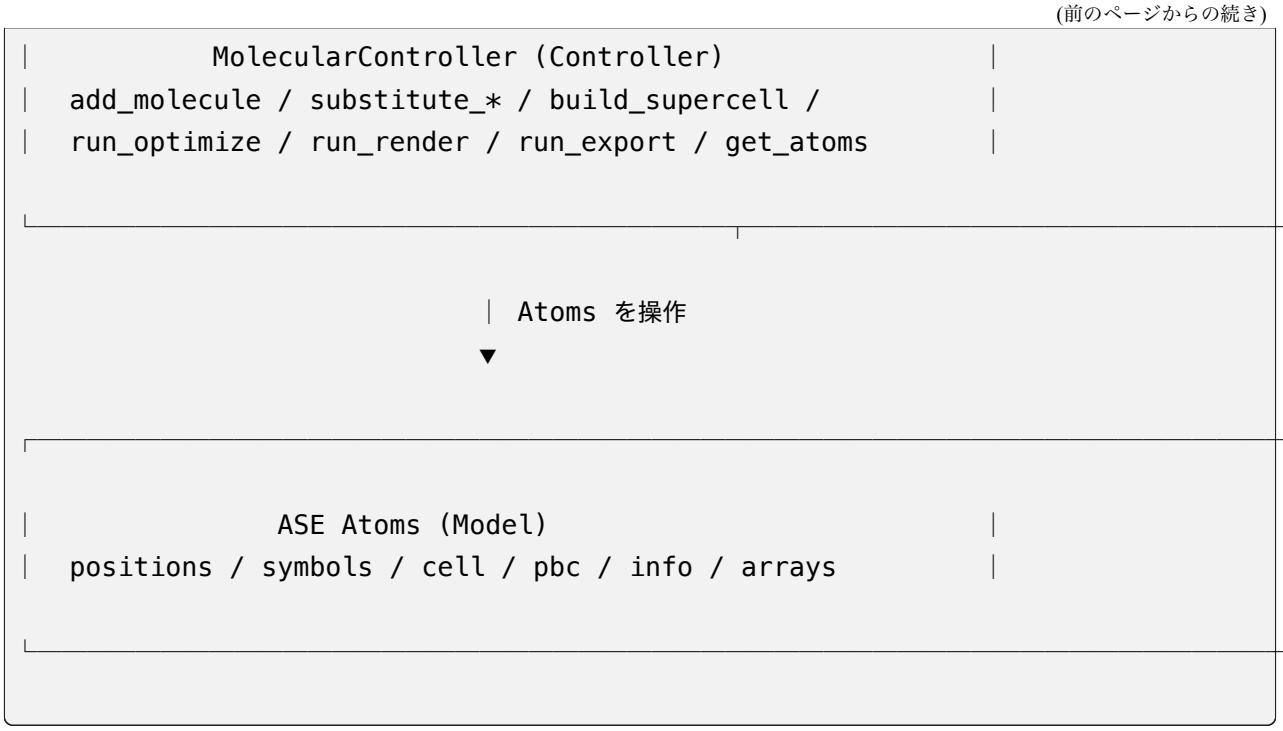
(次のページに続く)

(前のページからの続き)



(次のページに続く)

(前のページからの続き)



GUI モジュール構成:

- molecular_modeling_gui.py: メインの GUI クラス（ASE-GUI を拡張)。ウィンドウレイアウト、メニュー、ツールバー、サイドパネル、ステータスバーを管理
- source/gui/controller.py: GUI 操作をバックエンドに橋渡し
- source/gui/theme.py: テーマ設定（配色・フォント・余白の定数、ttk テーマ構成）
- source/gui/base_dialog.py: モーダルダイアログの基底クラス（OK/Cancel, バリデーション, 座標入力）
- source/gui/molecule_registry.py: data/*.toml から分子定義を動的に読み込み
- source/gui/add_molecule_dialog.py: 分子追加ダイアログ
- source/gui/substitution_dialogs.py: 4 種類の置換ダイアログ + 一括元素置換ダイアログ
- source/gui/export_dialog.py: エクスポートダイアログ
- source/gui/save_molecule_dialog.py: 選択原子の分子保存ダイアログ
- source/gui/molecule_list_dialog.py: 分子一覧ダイアログ
- source/gui/supercell_dialog.py: スーパーセル設定ダイアログ

付録

C

API リファレンス

このセクションでは、主要なモジュールと関数の API ドキュメントを自動生成しています。

C.1 モジュール構成

本ツールキットは以下のモジュール群で構成されています:

- **パイプライン**: `main` がエントリポイント。`build_system_from_toml` → `build_molecule_groups` → `optimize_voronoi_overlap` → `render_voronoi_overlays` → `write_structure` の順で処理
- **幾何演算**: `voronoi_geom` が Voronoi セルの構築・体積計算・IoU 評価を担当。`Polyhedron` クラスが中核
- **GUI**: `gui.controller` がビュー（ダイアログ）とモデル（ASE Atoms）を橋渡し。`gui.molecule_registry` が `data/*.toml` の動的読み込みを管理

プログラムから利用する場合の基本的な流れ:

```
from molecular_modeling.main import main

# input/input.toml を読み込んでパイプライン全体を実行
main()
```

C.2 メインパイプライン

C.2.1 main

`molecular_modeling.main.`**`main`**`()`

Entry point: build system, optimize orientations, render, and export structures.

Pipeline

1. Build *Atoms* from TOML inputs.
2. Group atoms by molecule/site.
3. Optimize Voronoi IoU via rotations (per site).
4. Check Voronoi volume tiling consistency.
5. Render overlays to PNGs.
6. Write QE/VASP/CIF outputs.

C.3 システム構築

C.3.1 build_system_from_toml

molecular_modeling.build_system_from_toml.**build_system_from_toml**(** (Keyword-only parameters separator (PEP 3102))*, *verbose: bool = True*) → Atoms

Build an ASE *Atoms* object from TOML inputs (supercell + substitutions).

パラメータ

verbose (*bool, optional*) -- If True, print a summary of the assembled system.

戻り値

Periodic supercell with per-atom *mol_id* and metadata in *system.info* (*groups*, *site_mol_name*, *dict_meta*, *rng*).

戻り値の型

ase.Atoms

メモ

Input (*input/input.toml*)

- *use_fractional=true* interprets *center_of_mass* as fractional and converts to Cartesian via *cell* (rows a,b,c).
- Downstream code always uses Cartesian site COMs.

Dictionary (*data/<name>.toml*)

- *voronoi_ref_id* selects which molecule COM is the reference (0-based).

- If *use_fractional=true*, both *positions* and *center_of_mass* are converted to Cartesian with that dictionary's *lattice_vectors*.
- *dict_meta* stores dictionary COMs in Cartesian.

Placement

- For each site: recenter *positions* by its COM, rotate (initially zero), then translate to the site COM; lattice tiling handled by *supercell_repeat*.

C.3.2 build_molecule_groups

molecular_modeling.build_molecule_groups.**build_molecule_groups**(*system: Atoms*) → List[List[int]]

Build atom index groups keyed by the *mol_id* array.

パラメータ

system (*ase.Atoms*) -- System holding *system.arrays['mol_id']*, where each value marks the placement/site identifier for an atom.

戻り値

List of atom index lists, ordered by *mol_id* (0..K-1).

戻り値の型

list[list[int]]

例外

ValueError -- If *mol_id* is missing from *system.arrays*.

C.4 Voronoi ジオメトリ

C.4.1 voronoi_geom

class molecular_modeling.voronoi_geom.**Polyhedron**(*verts: ndarray*, *faces: ndarray*)

ベースクラス: object

Minimal container for a convex polyhedron.

パラメータ

- **verts** (*ndarray*) -- Vertices array with shape (V, 3).
- **faces** (*ndarray*) -- Faces as indices into *verts* (F,3) or (F,k>=3).

verts

Vertex coordinates (float64).

Type
ndarray

faces

Face indices (int).

Type
ndarray

molecular_modeling.voronoi_geom.**polyhedron_to_trimesh**(*poly:* Polyhedron) → Trimesh

Convert *Polyhedron* to a processed *trimesh.Trimesh*.

パラメータ
poly (Polyhedron) -- Input polyhedron.

戻り値
Processed mesh with triangulated faces.

戻り値の型
trimesh.Trimesh

molecular_modeling.voronoi_geom.**compute_voronoi_cells_from_coms**(*cell: ndarray*, *coms_cart: ndarray*) → List[*Polyhedron*]

Compute periodic Voronoi cells for COM points and clip to the unit cell.

パラメータ

- **cell** (*ndarray*) -- Lattice vectors (3,3) with rows a, b, c (Cartesian Å).
- **coms_cart** (*ndarray*) -- COM coordinates (N,3) in Cartesian.

戻り値
Voronoi cells corresponding to each COM (same order).

戻り値の型
list[*Polyhedron*]

molecular_modeling.voronoi_geom.**rotate_and_place_polyhedron**(*poly:* Polyhedron, *euler_deg: Sequence[float]*, *src_center: ndarray*, *dst_center: ndarray*) → *Polyhedron*

Rotate a polyhedron about one center and translate to another.

パラメータ

- **poly** (Polyhedron) -- Polyhedron to transform.
- **euler_deg** (*sequence[float]*) -- Euler angles (XYZ, degrees).
- **src_center** (*ndarray*) -- Rotation center (3,).
- **dst_center** (*ndarray*) -- Target center after translation (3,).

戻り値

Rotated and translated polyhedron.

戻り値の型

Polyhedron

molecular_modeling.voronoi_geom.**compute_system_voronoi_cells**(*system: Atoms*, *groups: List[List[int]]*) → List[*Polyhedron*]

Compute Voronoi cells for system sites using atom groups.

パラメータ

- **system** (*ase.Atoms*) -- System with positions and lattice.
- **groups** (*list[list[int]]*) -- Atom indices per site.

戻り値

Voronoi cells per site (same order as *groups*).

戻り値の型

list[*Polyhedron*]

molecular_modeling.voronoi_geom.**rotate_polyhedron**(*poly: Polyhedron*, *euler_deg: Sequence[float]*, *center: ndarray*) → *Polyhedron*

Rotate a polyhedron about a specified center.

パラメータ

- **poly** (Polyhedron) -- Polyhedron to rotate.
- **euler_deg** (*sequence[float]*) -- Euler angles (XYZ, degrees).
- **center** (*ndarray*) -- Rotation center (3,).

戻り値

Rotated polyhedron.

戻り値の型

Polyhedron

molecular_modeling.voronoi_geom.**wrap_angles_deg**(*x: ndarray*) → ndarray

Wrap Euler angles to `[-180, 180)` for numerical stability.

パラメータ **x**

(*ndarray*) -- Angles in degrees.

戻り値

Wrapped angles.

戻り値の型

ndarray

molecular_modeling.voronoi_geom.**polyhedron_volume**(*poly:* Polyhedron) → float

Compute absolute volume of a closed polygonal mesh via tetrahedra sum.

パラメータ

poly (Polyhedron) -- Polyhedron with vertices and faces.

戻り値

Positive volume in Å^3.

戻り値の型

float

molecular_modeling.voronoi_geom.**overlap_volume_union_cart**(*A_sys: ndarray*, *coms_sys_cart: ndarray*, *A_dict: ndarray*, *coms_dict_cart: ndarray*, *angles_deg: Sequence[float]*, *p_ref_cart: ndarray*, *rng: int = 1*, *scale: float = 1.0*, *ref_idx: int = 0*) → float

Compute overlap volume between system and dictionary Voronoi cells.

パラメータ

- **A_sys** (*ndarray*) -- System lattice (3,3).
- **coms_sys_cart** (*ndarray*) -- System COMs (N,3).
- **A_dict** (*ndarray*) -- Dictionary lattice (3,3).
- **coms_dict_cart** (*ndarray*) -- Dictionary COMs (K,3).
- **angles_deg** (*sequence[float]*) -- Euler angles (XYZ, degrees) applied to dictionary.
- **p_ref_cart** (*ndarray*) -- Reference system point (3,).

- **rng** (*int, optional*) -- Replication radius.
- **scale** (*float, optional*) -- Uniform scaling of dictionary lattice/COMs.
- **ref_idx** (*int, optional*) -- Reference COM index in the dictionary.

戻り値

Overlap volume in Å^3.

戻り値の型

float

molecular_modeling.voronoi_geom.**single_set_cell_volume_cart**(*A_cell: ndarray*, *coms_cart: ndarray*, *p_ref_cart: ndarray*, *which: str*, *angles_deg: Sequence[float] | None = None*, *rng: int = 1*, *scale: float = 1.0*, *ref_idx: int = 0*) → float

Compute Voronoi cell volume for either system or dictionary seeds.

パラメータ

- **A_cell** (*ndarray*) -- Lattice vectors (3,3).
- **coms_cart** (*ndarray*) -- COMs (N,3).
- **p_ref_cart** (*ndarray*) -- Reference point (3,).
- **which** (*{'system', 'dict'}*) -- Whether to treat seeds as system or dictionary.
- **angles_deg** (*sequence[float], optional*) -- Euler angles for dictionary case.
- **rng** (*int, optional*) -- Replication radius.
- **scale** (*float, optional*) -- Uniform scaling (dictionary case).
- **ref_idx** (*int, optional*) -- Reference COM index (dictionary case).

戻り値

Voronoi cell volume in Å^3.

戻り値の型

float

例外

ValueError -- If *which* is not 'system' or 'dict'.

molecular_modeling.voronoi_geom.**iou_via_union_cart**(*A_sys: ndarray*, *coms_sys_cart: ndarray*, *A_dict: ndarray*, *coms_dict_cart: ndarray*, *p_ref_cart: ndarray*, *angles_deg: Sequence[float]*, *rng: int = 1*, *scale: float = 1.0*, *ref_idx: int = 0*) → float

Compute IoU between system and dictionary Voronoi cells at a reference site.

パラメータ

- **A_sys** (*ndarray*) -- System lattice (3,3).
- **coms_sys_cart** (*ndarray*) -- System COMs (N,3).
- **A_dict** (*ndarray*) -- Dictionary lattice (3,3).
- **coms_dict_cart** (*ndarray*) -- Dictionary COMs (K,3).
- **p_ref_cart** (*ndarray*) -- Reference system point (3,).
- **angles_deg** (*sequence[float]*) -- Euler angles (XYZ, degrees) for dictionary rotation.
- **rng** (*int, optional*) -- Replication radius.
- **scale** (*float, optional*) -- Uniform scaling for dictionary.
- **ref_idx** (*int, optional*) -- Reference COM index in the dictionary.

戻り値

Intersection-over-union in [0,1].

戻り値の型

float

molecular_modeling.voronoi_geom.**optimize_angles_each_system_site**(*A_sys: ndarray*, *coms_sys_cart: ndarray*, *A_dict: ndarray*, *coms_dict_cart: ndarray*, *sites_sys_cart: ndarray*, *angles_init: Sequence[float] = (0.0, 0.0, 0.0)*, *rng: int = 1*, *optimizer: str = 'Powell'*) → List[tuple]

Optimize dictionary orientation for each system site independently.

パラメータ

- **A_sys** (*ndarray*) -- System lattice (3,3).
- **coms_sys_cart** (*ndarray*) -- System COMs (N,3).
- **A_dict** (*ndarray*) -- Dictionary lattice (3,3).
- **coms_dict_cart** (*ndarray*) -- Dictionary COMs (K,3).
- **sites_sys_cart** (*ndarray*) -- Reference system sites (N,3).
- **angles_init** (*sequence[float], optional*) -- Initial Euler angles (XYZ, degrees) for all sites.
- **rng** (*int, optional*) -- Replication radius.
- **optimizer** (*str, optional*) -- SciPy optimizer method (passed to *minimize*).

戻り値

Tuples of `(site_index, angles_opt, best_iou)`.

戻り値の型

list[tuple]

molecular_modeling.voronoi_geom.**check_system_voronoi_volume_tiling**(*system: Atoms*, *groups: List[List[int]] | None = None*, ***, *rel_tol: float = 0.001*, *abs_tol: float = 1e-06*, *verbose: bool = True*) → dict

Check whether Voronoi cell volumes tile the supercell volume.

パラメータ

- **system** (*ase.Atoms*) -- System with positions and lattice.
- **groups** (*list[list[int]], optional*) -- Atom index groups per site. If None, computed from *mol_id*.
- **rel_tol** (*float, optional*) -- Relative tolerance for volume equality.
- **abs_tol** (*float, optional*) -- Absolute tolerance for volume equality.

- **verbose** (*bool, optional*) -- If True, print a report.

戻り値

Summary with keys: *sum_voronoi_volume*, *cell_volume*, *abs_diff*, *rel_error*, *tolerances*, *ok*, *site_volumes*.

戻り値の型

dict

C.5 最適化

C.5.1 optimize_voronoi_overlap

molecular_modeling.optimize_voronoi_overlap.**optimize_voronoi_overlap**(*system: Atoms*, *groups: List[List[int]] | None = None*, *x0_euler_deg: ndarray | None = None*, ***, *select_sites: Sequence[int] | None = None*, *grid_num: Tuple[int, int, int] = (13, 13, 13)*, *angle_bounds: Tuple[float, float] = (-180.0, 180.0)*, *rng: int | None = None*, *log_dir: str | Path | None = None*) → Atoms

Optimize per-site Euler angles to maximize Voronoi IoU with uniform scaling.

パラメータ

- **system** (*ase.Atoms*) -- Input supercell with *mol_id* array and dictionary metadata in *system.info*.
- **groups** (*list[list[int]], optional*) -- Atom index groups per site. If None, computed via *build_molecule_groups*.
- **x0_euler_deg** (*ndarray, optional*) -- Initial Euler angles (N,3) in degrees. Defaults to zeros or *system.info["site_angles_deg"]*.
- **select_sites** (*sequence[int], optional*) -- Subset of site indices to optimize. Others remain unchanged.
- **grid_num** (*tuple[int, int, int], optional*) -- Grid counts for coarse search over (alpha, beta, gamma).
- **angle_bounds** (*tuple[float, float], optional*) -- Inclusive bounds for grid search angles in degrees.
- **rng** (*int, optional*) -- Random seed for Voronoi sampling; falls back to *system.info["rng"]*.
- **log_dir** (*str or Path, optional*) -- If provided, writes per-site optimization logs to the directory.

戻り値

The same *Atoms* with positions rotated/scaled in place and optimization results stored in *system.info* (*optimized_angles_deg*, *optimized_scales*, *optimized_iou*).

戻り値の型

ase.Atoms

メモ

Per-site procedure:

1. Compute system Voronoi cell volume `V_sys` at the site COM.
2. Compute dictionary Voronoi cell volume `V_dict` (rotation-invariant).
3. Set uniform scale `s = (V_sys / V_dict)^(1/3)` (or 1 if `V_dict <= 0`).
4. Grid search then Nelder–Mead to maximize IoU using rotated+scaled dictionary seeds.
5. Apply rotation and uniform scaling about the site COM to atom coordinates.

C.6 可視化

C.6.1 render_voronoi_overlays

molecular_modeling.render_voronoi_overlays.**render_voronoi_overlays**(*system: Atoms*, **, groups: List[List[int]] | None = None*, *angles_deg: ndarray | None = None*, *sites: Sequence[int] | None = None*, *out_dir: str | Path = './voronoi_pngs'*, *dpi: int = 220*, *figsize: Tuple[float, float] = (6.0, 5.0)*, *azim: float = -60.0*, *elev: float = 20.0*, *rng: int | None = None*, *show_overlap: bool = True*) → None

Render Voronoi overlays for selected sites and report volumes.

パラメータ

- **system** (*ase.Atoms*) -- System with *system.info* containing *site_mol_name*, *dict_meta*, and optional *optimized_angles_deg* / *optimized_scales*.
- **groups** (*list[list[int]], optional*) -- Atom index groups per site. If None, computed with *build_molecule_groups*.
- **angles_deg** (*ndarray, optional*) -- Euler angles (N,3) in degrees; defaults to optimized values or zeros.
- **sites** (*sequence[int], optional*) -- Site indices to render; defaults to all.
- **out_dir** (*str or Path, optional*) -- Output directory for PNGs.

- **dpi** (*int, optional*) -- Figure DPI.
- **figsize** (*tuple[float, float], optional*) -- Matplotlib figure size in inches.
- **azim** (*float, optional*) -- Azimuth angle for 3D view.
- **elev** (*float, optional*) -- Elevation angle for 3D view.
- **rng** (*int, optional*) -- Random seed for Voronoi sampling; falls back to *system.info["rng"]*.
- **show_overlap** (*bool, optional*) -- If True, plot overlap polyhedron in orange.

C.7 出力

C.7.1 write_structure

molecular_modeling.write_structure.**write_structure**(*system: Atoms*, *, *outdir: str | Path = '.'*, *verbose: bool = True*)

Write simulator-friendly structure files from an ASE *Atoms* object.

パラメータ

- **system** (*ase.Atoms*) -- System to export.
- **outdir** (*str or Path, optional*) -- Output directory. Created if missing.
- **verbose** (*bool, optional*) -- If True, print a summary of written files.
- **Writes**
- **------**
- **input** (*– Quantum ESPRESSO*)
- **inputs** (*– VASP*)
- **CIF** (*–*)

C.8 GUI

C.8.1 controller

Controller for GUI-backend integration.

Provides high-level operations that bridge GUI dialogs with the molecular modeling backend.

class molecular_modeling.gui.controller.**MolecularController**(*atoms: Atoms | None = None*)

ベースクラス: object

Controller for molecular modeling operations.

Bridges GUI dialogs with the molecular modeling backend operations.

パラメータ
atoms (*ase.Atoms, optional*) -- Initial atoms object to operate on.

set_atoms(*atoms: Atoms*) → None

Set the current atoms object.

get_atoms() → Atoms | None

Get the current atoms object.

add_molecule(*molecule_name: str*, *position: Tuple[float, float, float]*, *rotation: Tuple[float, float, float] = (0.0, 0.0, 0.0)*) → bool

Add a molecule to the current system.

パラメータ

- **molecule_name** (*str*) -- Name of the molecule to add.
- **position** (*tuple[float, float, float]*) -- Center of mass position in Cartesian coordinates.
- **rotation** (*tuple[float, float, float], optional*) -- Euler angles (XYZ, degrees) for rotation.

戻り値
True if molecule was added successfully.

戻り値の型
bool

substitute_molecule(*molecule_index: int*, *new_molecule_name: str*) → bool

Replace a molecule at a specific index with a different molecule.

Preserves the orientation of the original molecule using principal component analysis.

パラメータ

- **molecule_index** (*int*) -- Index of the molecule group to replace.
- **new_molecule_name** (*str*) -- Name of the new molecule to insert.

戻り値
True if substitution was successful.

戻り値の型
bool

global_atom_substitution(*old_symbol: str*, *new_symbol: str*) → int

Replace all atoms of one element with another.

パラメータ

- **old_symbol** (*str*) -- Element symbol to replace.
- **new_symbol** (*str*) -- New element symbol.

戻り値

Number of atoms replaced.

戻り値の型

int

substitute_atom(*atom_index: int*, *new_symbol: str*) → bool

Replace a single atom with a new element.

パラメータ

- **atom_index** (*int*) -- Index of the atom to replace.
- **new_symbol** (*str*) -- New element symbol.

戻り値

True if substitution was successful.

戻り値の型

bool

substitute_molecule_atoms(*molecule_index: int*, *new_symbols: List[str]*) → Tuple[bool, str]

Replace all atoms in a molecule with new symbols.

パラメータ

- **molecule_index** (*int*) -- Index of the molecule group.
- **new_symbols** (*List[str]*) -- New element symbols (can be compact notation like "C6H6").

戻り値

(success, error_message). error_message is empty if success.

戻り値の型

tuple[bool, str]

set_supercell_repeat(*repeats: List[int]*) → None

Set the supercell repeat factors.

get_supercell_repeat() → List[int]

Get the current supercell repeat factors.

set_use_fractional(*use_fractional: bool*) → None

Set whether to use fractional coordinates.

get_use_fractional() → bool

Get whether fractional coordinates are used.

get_molecule_groups() → List[Dict]

Get information about all molecule groups in the system.

戻り値

List of dictionaries with molecule group information.

戻り値の型

list[dict]

get_selected_molecule_index(*selected_atoms: List[int]*) → int | None

Get the molecule index for a set of selected atoms.

パラメータ

selected_atoms (*list[int]*) -- List of selected atom indices.

戻り値

Molecule index if all selected atoms belong to the same molecule.

戻り値の型

int or None

molecular_modeling.gui.controller.**get_controller**() → *MolecularController*

Get the global controller instance.

戻り値

The global controller instance.

戻り値の型

MolecularController

molecular_modeling.gui.controller.**reset_controller**() → *MolecularController*

Reset and return a fresh global controller instance.

Use this when starting a new GUI session to avoid stale state.

戻り値

fresh controller instance.

A

戻り値の型

MolecularController

C.8.2 molecule_registry

Molecule registry for dynamic loading of molecule definitions from TOML files.

Scans the `data/*.toml` directory and provides access to molecule configurations.

class molecular_modeling.gui.molecule_registry.**MoleculeRegistry**(*data_dir: str | Path | None = None*)

ベースクラス: `object`

Registry for molecule definitions loaded from TOML files.

Scans the data/ directory for .toml files and loads molecule configurations.

パラメータ

data_dir (*str or Path, optional*) -- Path to the directory containing molecule TOML files. Defaults to "data" relative to the package root.

molecules

Dictionary mapping molecule names to their configurations.

Type

dict[str, dict]

reload() → None

Reload all molecule definitions from disk.

get_molecule_names() → List[str]

Get list of available molecule names.

戻り値

Sorted list of molecule names.

戻り値の型

list[str]

get_molecule(*name: str*) → dict | None

Get molecule configuration by name.

パラメータ

name (*str*) -- Molecule name.

戻り値

Molecule configuration, or None if not found.

戻り値の型

dict or None

get_symbols(*name: str*) → List[str]

Get expanded atomic symbols for a molecule.

パラメータ

name (*str*) -- Molecule name.

戻り値

List of atomic symbols (expanded from compact notation).

戻り値の型

list[str]

get_positions(*name: str*) → ndarray

Get atomic positions for a molecule.

パラメータ

name (*str*) -- Molecule name.

戻り値

Atomic positions as (N, 3) array.

戻り値の型

numpy.ndarray

get_num_atoms(*name: str*) → int

Get number of atoms in a molecule.

パラメータ

name (*str*) -- Molecule name.

戻り値

Number of atoms.

戻り値の型

int

get_unique_elements(*name: str*) → List[str]

Get unique elements in a molecule.

パラメータ

name (*str*) -- Molecule name.

戻り値

Sorted list of unique element symbols.

戻り値の型

list[str]

molecular_modeling.gui.molecule_registry.**get_registry**() → *MoleculeRegistry*

Get the global molecule registry instance.

戻り値

The global registry instance.

戻り値の型

MoleculeRegistry